Evasion Games: Dissecting Malware Evasive Techniques Across Packers, Sandboxes, and Debuggers

DANIELE MAMMONE, Politecnico di Milano, Italy
FRANCESCO SIDOTI MIGLIORE, Politecnico di Milano, Italy
STEFANO LONGARI, Politecnico di Milano, Italy
MICHELE CARMINATI, Politecnico di Milano, Italy
STEFANO ZANERO, Politecnico di Milano, Italy
MARIO POLINO, Politecnico di Milano, Italy

Malware authors increasingly leverage evasive techniques to conceal malicious behavior from dynamic analysis tools. Despite prior efforts to categorize such techniques, the resilience of existing analysis environments—sandboxes and debuggers—against them remains underexplored. In this work, we provide an updated and extended taxonomy of evasive techniques, identifying 55 newly documented methods. We then evaluate the extent to which 25 commercial and publicly available packers embed evasive behaviors into obfuscated binaries. Finally, we assess the readiness of seven online sandboxes and four debuggers in handling these techniques. Our experiments reveal a significant disparity: while sandboxes generally exhibit higher readiness against anti-debugging and anti-virtualization techniques, they are still vulnerable to miscellaneous evasions, particularly timing-based attacks. Conversely, debuggers prove less effective overall, with only 27% of evasions successfully countered. These findings underscore the persistent limitations of current analysis environments and motivate the development of more robust anti-evasion strategies.

CCS Concepts: • **Security and privacy → Malware and its mitigation**.

Additional Key Words and Phrases: Malware analysis, dynamic analysis, evasive software, sandbox

ACM Reference Format:
Daniele Mammone, Francesco Sidoti Migliore, Stefano Longari, Michele Carminati, Stefano Zanero, and Mario Polino. 2026. Evasion Games: Dissecting Malware Evasive Techniques Across Packers, Sandboxes, and Debuggers. *ACM Trans. Priv. Sec.* 29, 4, Article 45 (November 2026), 31 pages. https://doi.org/10.1145/3846178

1 Introduction

Malware is one of the most significant challenges in computer security. Its spread continues to grow, with cybercriminals increasingly targeting individuals, businesses, and government agencies through various forms of malware, such as ransomware, banking trojans, and botnets [11]. To counter these threats, the research community has developed tools to detect malware, based on

Authors' Contact Information: Daniele Mammone (corresponding author), DEIB - Department of Electronics, Informatics and Bioengineering, Politecnico di Milano, Milan, Milan, Italy; e-mail: daniele.mammone@polimi.it; Francesco Sidoti Migliore, Politecnico di Milano, Milan, Lombardy, Italy; e-mail: francesco.sidoti@mail.polimi.it; Stefano Longari, DEIB, Politecnico di Milano, Milano, Italy; e-mail: stefano.longari@polimi.it; Michele Carminati, DEIB - Department of Electronics, Informatics and Bioengineering, Politecnico di Milano, Milan, Milan, Italy; e-mail: michele.carminati@polimi.it; Stefano Zanero, DEIB - Department of Electronics, Informatics and Bioengineering, Politecnico di Milano, Milan, Milan, Italy; e-mail: stefano.zanero@polimi.it; Mario Polino, DEIB - Department of Electronics, Informatics and Bioengineering, Politecnico di Milano, Milan, Milan, Italy; e-mail: mario.polino@polimi.it.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2471-2566/2026/11-ART45
https://doi.org/10.1145/3846178

two principal approaches, static and dynamic analysis [21]. Static analysis detects malware by examining its static content, such as program bytes or binary structures, while dynamic analysis identifies malicious patterns by observing the program's execution behavior. However, detecting malware statically has become increasingly challenging, if not impossible. Advanced obfuscation techniques like polymorphism [71] and metamorphism [80] have rendered many static tools ineffective. Polymorphic malware dynamically alters its code with each iteration, and metamorphic malware rewrites its code entirely, defeating automatic detection systems that rely on known byte sequences. These techniques also complicate manual analysis, significantly increasing the effort required to understand a program's behavior using reverse engineering tools [46]. Dynamic analysis, on the other hand, has emerged as a more robust alternative, as it observes the behavior of malware during execution [1]. Manual dynamic analysis involves executing a program within a debugger, allowing an analyst to monitor its actions and system interactions step by step. However, this method is time-consuming and resource-intensive. To address these challenges, researchers have shifted their focus toward automated dynamic analysis methodologies. Sandboxes have become essential tools in this effort, providing isolated environments where programs can be executed safely and monitored in detail [73]. These environments capture comprehensive behavioral data, including network traffic, executed commands, and interactions with operating system APIs. Data are then analyzed using external tools, such as machine learning models, to determine whether the observed behavior is malicious. However, malware authors have developed sophisticated anti-analysis techniques to detect and evade dynamic analysis environments [24]. The effectiveness of dynamic analysis tools is therefore based on their ability to handle advanced evasion techniques, hiding artifacts which may be checked by malware to recognize whether they are being analyzed. Malware authors exploit the inherent detectability of dynamic environments, designing malware to identify such setups during execution. If malware detects signs of analysis, it can stop execution, exhibit benign behavior, or adapt its operations to avoid detection. In contrast, if the environment appears legitimate (i.e., environments intended for genuine use rather than for analysis), the malware proceeds with its malicious activity. To counter these sophisticated evasion strategies, modern analysis environments must replicate legitimate system behavior, e.g., simulating user interactions, generating realistic system activity, and obfuscating analysis-related artifacts such as virtual machine indicators or debugger processes. To this direction, the research community moved to study evasive behaviors and produced taxonomies for Windows [24] and Android malware [60], identifying which are the most common evasive techniques, to develop countermeasures inside analysis environments. Other works focused on the development of sandboxes or debuggers hiding analysis artifacts, allowing malware to expose their behavior during analysis.

No work, to the best of our knowledge, has yet developed an extensive evaluation of the coverage of analysis environments in handling evasive techniques, assessing their ability to counter evasion strategies. To the best of our knowledge, we are the first to perform such an assessment. Moreover, since many malware authors do not directly protect their malware, but rely on commercial packers to obfuscate them or to protect them against dynamic analysis environments, we extend our evaluation to analyze the techniques used by these tools. To achieve our objective, we start updating current taxonomies of evasive techniques. Then, we evaluate the evasive techniques introduced in the generated obfuscated binaries by twenty-five publicly available packers. We finally test seven publicly accessible online sandboxes and four debuggers for the Windows operating system to determine whether they actively conceal analysis artifacts or if the evasive techniques are effective in detecting their presence, with a particular attention to those implemented by packers. The results of our experiments show that debuggers are not yet effective in countering anti-debugging evasion techniques, successfully countering only about 27% of them. However, they appear more capable when addressing miscellaneous and anti-virtualization techniques, achieving an average

success rate of 85% for both. In contrast, sandboxes are more effective at countering anti-debugging and anti-virtualization techniques, successfully handling an average of 69% and 57% of them, respectively. However, they prove more fragile against miscellaneous techniques, with only 42% being successfully countered.

Our contributions are therefore as follows:

— We extend and update previous taxonomies on evasive techniques with 55 novel techniques over a total of 191.
— We identify the evasive techniques mostly used by 25 available packers.
— We study the anti-evasion capabilities implemented by 7 online sandboxes and 4 debuggers against the updated taxonomy.
— We identify the evasive techniques implemented by packers that are most effective against the studied analysis environments.

The rest of the article is structured as follows. Section 2 provides the necessary technical background for understanding this work. In Section 3, we identify the research questions we want to answer. In Section 4, we present the updated taxonomy, outlining the different categories and highlighting the newly identified techniques. Section 5 describes the methodology used for testing. Finally, in Section 6, we report the results of the experiments we conducted against packers, sandboxes and debuggers.

2 Background on Evasive Techniques and Analysis Methods

In this section, we introduce the background necessary to understand the contents of this article. First, we describe the two main approaches to malware analysis—static and dynamic—and outline the available tools associated with each approach. Finally, we introduce what is an evasive software, and why it is relevant studying countermeasures against evasive techniques in the context of malware analysis.

2.1 Static and Dynamic Analysis

Static and dynamic analysis are the two different approaches used to do reverse engineering of compiled binaries, e.g., programs for which the original source code is unavailable. In this section, we introduce the technical details of static and dynamic analysis.

2.1.1 Static Analysis. It is a collection of program analysis techniques that examine programs without executing them, relying instead on information extracted directly from their byte sequences. A classic example is signature analysis, which checks binary files against known malware signatures—distinct byte sequences characteristic of known threats—to classify them as malicious or benign [5]. Beyond simple signature matching, static analysis includes more sophisticated methods that aim at extracting deeper semantic information. These include identifying individual functions within binaries, generating pseudo-C code representations, computing **Control Flow Graphs (CFG)** [3], and performing Taint Analysis to track data propagation through a program [7]. By leveraging semantic understanding, such advanced techniques significantly enhance malware detection accuracy compared with basic signature-based methods [8, 76]. However, despite these advancements, static analysis methods remain vulnerable to obfuscation techniques [46]. Malware authors commonly use polymorphism—a technique that encrypts or otherwise obfuscates executable bytes, revealing their true form only during runtime [71]—and metamorphism, which generates multiple functionally identical malware variants with entirely different bytecode patterns [80]. The analysis challenges imposed by obfuscation methods led researchers to explore novel dynamic analysis techniques capable of overcoming the inherent limits of static approaches.

2.1.2 Dynamic Analysis. Contrary to static analysis, dynamic analysis techniques analyze a binary executable by executing it and extracting its runtime behavior. In the malware detection task, such information is used to determine whether a program is goodware or malware by analyzing its dynamic behavior.

Debuggers are software tools used by developers and security analysts to examine, analyze, and manipulate the execution of malware. They use APIs provided by the underlying operating system to monitor and control execution, such as accessing memory, stopping execution at specific points (i.e., breakpoints), intercepting signals, and tracing calls to OS APIs. The most well-known Windows debuggers include WinDbg [14], OllyDbg [78], Immunity Debugger [26], x64dbg [74], and HyperDbg [33]. On Linux, the most widely used debuggers are LLDB [37] and GDB [25], while more recent efforts such as the libdebug project [19] aim at introducing greater scriptability and configurability into debugging workflows. **Dynamic Binary Instrumentation (DBI)** is a technique allowing the analysis and modification of a binary's behavior at runtime. DBI inserts custom instrumentation code into a running program without requiring access to its source code. The instrumentation code can be used to extract the program's runtime behavior. The most notable DBI frameworks are **Pin** [40], **Valgrind** [49], and **Frida** [54]. **Valgrind** performs its analysis by dynamically translating the binary's machine code into an intermediate representation (IR) called VEX, which is independent of the underlying hardware architecture. Valgrind then applies instrumentation to the resulting IR before **just-in-time** (**JIT**) recompiling it back into machine code. **Pin** offers two modes of instrumentation: **JIT Mode** and **Probe Mode**. JIT Mode uses a *just-in-time* compiler to dynamically recompile the code under execution and insert instrumentation. Probe Mode hooks function calls using probes, a technique that replaces the first instruction of a routine with an unconditional jump to the replacement routine. **Frida** provides robust scripting capabilities via JavaScript and Python, enabling rapid prototyping and adaptation to different environments. It operates with minimal overhead and supports both user-mode and kernel-mode introspection across multiple platforms.

Another widely used runtime analysis technique is **API hooking** [38], which allows intercepting function calls within an application to monitor, modify, or suppress their execution. This technique is commonly employed in malware analysis, security monitoring, and reverse engineering. API hooking can be implemented using several methods, including inline hooking, **Import Address Table** (**IAT**) hooking, and **Export Address Table** (**EAT**) hooking. Inline hooking involves modifying the first few bytes of a function to redirect execution to a custom handler, while IAT and EAT hooking manipulate the tables used by Windows to resolve dynamically linked function calls. Tools like Microsoft Detours, MinHook, and Frida facilitate API hooking, allowing analysts to inspect interactions between applications and the operating system in real-time.

Sandboxes are automated tools that execute a program inside an isolated environment and use dynamic techniques, such as DBI, debugging, or API hooking, to extract the program's behavior, e.g., network traffic and OS API usage. This information is used to determine whether a sample is malicious by feeding it into AI-based malware classifiers [66]. All the tools introduced in this section, however, leave artifacts in the system that are detectable by malware before execution. Debuggers modify certain fields in the memory of the process under analysis, while sandboxes can be easily fingerprinted by querying information about the running CPU.

2.2 Evasive Software

Evasive software checks for the presence of artifacts to determine if it is being analyzed, e.g., it is running in a virtualized environment, a sandbox, or under a debugger. Depending on the outcome of these checks, it may alter its behavior, e.g., terminating early. Such evasion strategies are employed by both commercial software, to prevent reverse engineering of copyrighted code,

and by malware. The latter want to hide malicious behavior from dynamic analysis tools, which could otherwise generate behavioral fingerprints to support static detection methods.

3 Motivation

Despite significant research efforts to understand and classify evasive behaviors in malware, most studies have focused on taxonomies or the development of sandboxes and debuggers designed to evade detection. Our objective is to evaluate the effectiveness of analysis environments in detecting and mitigating evasive techniques, with particular emphasis on those employed by packers, which can automate the evasion process. To achieve our objective, we define a set of research questions that we aim at answering in this article:

- **RQ1**: What techniques have been introduced since the publication of the previously published taxonomies of evasive techniques?
- **RQ2**: What are the evasive techniques added to obfuscated binaries by available commercial packers in the resulting obfuscated binaries?
- **RQ3**: What is the coverage of publicly available analysis environments (i.e., sandboxes and debuggers) in handling evasive techniques?
- **RQ4**: What is the effectiveness of techniques added by packers against the tested analysis environments?

We answer to RQ1 in Section 4, to RQ2 in Section 6.1, to RQ3 and RQ4 in Sections 6.2 and 6.3.

4 Taxonomy

In this section, we introduce the taxonomy used to conduct our study and present our contributions to updating it from previous work. The previous state of the art taxonomy on evasive techniques is the one proposed by Galloro et al.[24]. Even being the most updated systematization about evasive malware, it covers evasive techniques discovered until 2020. Following their search methodology, we identify 55, over a total of 191, new evasive techniques and coherently update the systematization to better suit the new introduced techniques. These techniques are marked in bold in Table 1.

With respect to previous works, we introduce two new families of evasive techniques, **Geofencing** and **Emulation**. Geofencing techniques detect analysis environments by checking locale settings, where unusual locale settings are indicators of an analysis environment. The **Emulation** family includes techniques checking for installed emulation software, e.g., QEMU. We further divide these families into macro-categories of techniques, depending on the use of *system artifacts*, *side-channel* measurements or the current execution behavior of the process to classify a system.

4.1 System Artifacts

This class of checks searches for system artifacts that may indicate the presence of an analysis environment. For example, it includes inspecting traces of human activity [45], such as recently accessed files, installed applications, or browser history. Since sandboxes typically start from a fresh Windows installation, such artifacts are often absent. Another approach consists in examining artifacts in the process memory, which may reveal information about the debugging status, the parent process, or other runtime properties [24]. Finally, evasive software may inspect system information to identify hardware components notoriously associated with hypervisors, such as specific CPU features or virtualized devices. More specifically, the families of techniques are:

- **System Environment**: utilize system API calls to query specific attributes and configurations of the OS, aiming to detect artifacts that indicate the presence of an analysis environment. This includes hardware components such as RAM, hard drives, the CPU, and network adapters, as well as software elements like the system's hostname.

Table 1. Experimental Results Across All Tested Environments

Category	References	Technique	T	P	S1	S2	S3	S4	S5	S6	S7	D1	D2	D3	D4
CPU Fingerprinting	[15] [64]	CPUID (EAX=0x00000001)	V	8	•	◦	◦	•	•	◦	◦	◦	◦	◦	◦
		IN	V	0	◦	◦	•	◦	◦	◦	◦	•	◦	•	◦
		CPUID (EAX=0x40000000)	V	1	◦	◦	•	•	•	◦	◦	◦	◦	◦	◦
		STR	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		SMSW	V	0	•	•	•	•	•	•	•	◦	◦	◦	◦
Drivers Information	[18]	EnumDeviceDrivers and GetDeviceDriverBaseName	V	0	◦	◦	•	◦	•	◦	◦	◦	◦	◦	◦
Exception Handling	[55] [22]	**DebugBreak**	D	0	◦	◦	◦	◦	•	◦	◦	•	•	•	•
		SetUnhandledExceptionFilter	D	16	◦	◦	◦	◦	•	◦	◦	•	•	•	•
		POPF/D - TRAP FLAG	D	7	◦	◦	◦	◦	•	◦	◦	-	-	•	•
		NtClose(INVALID_HANDLE)	D	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	•	•
		CTRL Exception Handling	D	0	◦	•	◦	◦	◦	•	◦	•	•	•	•
		OutputDebugString	D	1	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
Emulation	[39]	Check Virtual PC	V	0	•	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		Check QEMU	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		Check Xen	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		Check Wine	V	0	•	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
File System	[56] [61] [29] [27]	**GetModuleFileNameA**	V	8	◦	◦	◦	◦	◦	◦	•	•	•	•	•
		GetProcessImageFileNameA	V	5	◦	◦	◦	◦	◦	-	•	•	•	•	•
		QueryFullProcessImageName	V	0	◦	◦	◦	◦	◦	◦	•	•	•	•	•
		MsiGetProductInfoA	M	0	◦	◦	•	◦	•	-	•	◦	◦	◦	◦
		EnumerateRecentFiles	M	0	◦	◦	◦	◦	•	-	•	•	•	•	•
		NtOpenFile	M	1	◦	◦	◦	◦	◦	◦	•	◦	◦	◦	◦
		NtQueryAttributesFile	M	0	◦	◦	◦	◦	◦	◦	•	◦	◦	◦	◦
		NtCreateFile	M	1	◦	•	•	◦	•	•	•	◦	◦	◦	◦
		NtQueryDirectoryFileEx	M	0	◦	◦	•	•	◦	◦	•	◦	◦	◦	◦
Human Interaction	[42] [65] [23]	**GetForegroundWindow**	V	5	•	◦	◦	◦	•	•	◦	◦	◦	◦	◦
		GetAsyncKeyState	V	0	•	•	•	◦	•	•	•	◦	◦	◦	◦
		SetWindowsHookExA	V	1	•	•	•	◦	•	•	•	◦	◦	◦	◦
		MessageBox	V	0	•	•	◦	◦	•	•	◦	◦	◦	◦	◦
		GetCursorPos	V	2	•	◦	◦	◦	•	◦	•	◦	◦	◦	◦
		GetLastInputInfo	V	0	•	◦	◦	◦	◦	•	◦	◦	◦	◦	◦
		SetWindowsHookScrolling	V	0	•	•	◦	•	•	•	•	◦	◦	◦	◦
List Process	[27]	NtQSI(SYSTEM_PROCESS_INF)	M	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		EnumProcesses and GetModuleBaseName	M	1	◦	◦	◦	◦	•	◦	◦	◦	◦	◦	◦
		GetProcessIdFromName	D	0	•	◦	◦	◦	•	◦	◦	•	•	•	•
List Services	[27]	OpenSCManager and EnumServicesStatus	V	1	◦	◦	•	◦	◦	◦	◦	◦	◦	◦	◦
		OpenService	V	0	◦	◦	•	◦	◦	◦	◦	◦	◦	◦	◦
		GetServiceDisplayName	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GetServiceKeyName	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
System Environment	[15] [36] [43] [72] [64]	PEB->BeingDebugged	D	1	•	◦	◦	◦	◦	◦	◦	•	•	•	•
		PEB->NtGlobalFlag	D	0	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		PEB->Heap->Flags	D	0	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		PEB->Heap->ForceFlags	D	0	•	◦	◦	◦	◦	◦	◦	•	•	•	•
		KUSER_SHARED->KdDebuggerEnabled	D	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
Others	[56] [59]	**WNetGetProviderName**	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GetLogicalDriveType	V	2	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		NtSetDebugFilterState	D	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		IsParentExplorerExe	M	0	◦	•	◦	•	•	•	◦	•	•	•	•
		instructionPrefixes	D	0	◦	◦	◦	◦	•	◦	◦	•	•	•	•
		IsHooked	M	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		dbgExcp	D	0	◦	◦	◦	◦	◦	•	◦	•	•	•	•
		VirtualAlloc_WriteWatch_BufferOnly	M	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		VirtualAlloc_WriteWatch_APICalls	M	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		VirtualAlloc_WriteWatch_IsDebuggerPresent	D	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		VirtualAlloc_WriteWatch_CodeWrite	M	0	◦	◦	◦	•	◦	◦	◦	•	•	•	•
		hideDesktop	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		CanOpenCsrss	M	5	◦	◦	◦	◦	•	•	◦	◦	◦	◦	◦
		MemoryBreakpoints_PageGuard	D	0	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		NtYieldExecution	D	0	•	•	◦	◦	•	•	•	•	•	•	•
		SetHandleInformation_ProtectedHandle	D	0	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		query_license_value	V	0	◦	◦	◦	◦	•	◦	◦	◦	◦	◦	◦
		power_capabilities	V	0	◦	◦	◦	•	•	◦	◦	◦	◦	◦	◦
		pirated_windows	V	0	◦	◦	•	◦	◦	•	◦	•	•	•	•
		check_loadLibrary	M	0	◦	◦	◦	◦	◦	◦	◦	•	◦	•	◦

(Continued)

Table 1. Continued

Category	References	Technique	T	P	S1	S2	S3	S4	S5	S6	S7	D1	D2	D3	D4
Process	[56] [22],	**GetModuleHandle**	M	3	•	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
Environment	[35] [64]	NtSetContextThread$CONTEXT_DEBUG_REGISTERS)	D	0	•	◦	◦	◦	•	◦	•	•	•	•	•
		NtQueryInformationProcess(0x07)	D	3	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		NtSetInformationThread(0x11)	D	4	•	•	•	•	•	•	•	•	•	•	•
		NtQueryInformationProcess(0x1e)	D	0	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		NtQueryInformationProcess(0x1f)	D	2	◦	•	◦	◦	◦	◦	◦	•	•	•	•
		NtGetContextThread$CONTEXT_DEBUG_REGISTERS)	D	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		DebugActiveProcess on Parent	D	3	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		NtCreateThreadEx$HIDE_FROM_DEBUGGER)	D	0	•	◦	◦	◦	•	◦	◦	•	•	•	•
Registry	[27]	NtOpenKey/Ex	M	1	•	◦	•	•	◦	•	•	•	•	•	•
		NtEnumerateKey	M	0	◦	◦	◦	◦	•	◦	•	◦	◦	◦	◦
		NtQueryValueKey	M	0	◦	◦	◦	◦	◦	◦	•	◦	◦	◦	◦
		NtEnumerateValueKey	M	0	◦	◦	◦	◦	◦	◦	•	◦	◦	◦	◦
System	[39] [17] [62]	**GetUsernameA**	V	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
Environment	[47] [67] [16]	**GetSystemInfo**	V	2	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
	[68] [41] [61]	**IsNativeVhdBoot**	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
	[56] [31] [57]	**EnumWindows**	V	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
	[58] [13] [15]	**EnumDisplayMonitors**	V	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
	[36] [43]	**GetSystemMetrics**	V	8	◦	◦	◦	◦	•	•	◦	◦	◦	◦	◦
	[72] [64]	**GetDesktopWindowRect**	V	6	◦	◦	◦	◦	•	•	◦	◦	◦	◦	◦
		EnumDisplaySettings	V	0	◦	◦	◦	◦	•	•	◦	◦	◦	◦	◦
		GetProcessAffinityMask	V	2	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		SetupDi_DiskDrive	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		EnumPrinters	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GetSystemDefaultLangID	V	0	•	•	•	•	•	•	•	•	•	•	•
		GetUserDefaultLangID	V	4	•	•	•	•	•	•	•	•	•	•	•
		GetKeyboardLayoutList	V	5	•	•	•	•	•	•	•	•	•	•	•
		GetKeyboardLayout	V	5	•	•	•	•	•	•	•	•	•	•	•
		IsProcessorFeaturePresent	V	0	◦	•	◦	◦	◦	◦	◦	◦	◦	◦	◦
		NtSystemDebugControl	D	0	◦	◦	◦	◦	◦	◦	◦	-	-	◦	◦
		CreateMutex	D	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		OpenMutex	D	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		WudfIsAnyDebuggerPresent	D	0	◦	◦	◦	◦	◦	◦	◦	-	-	•	•
		WudfIsUserDebuggerPresent	D	0	◦	◦	◦	◦	◦	◦	◦	-	-	•	•
		WudfIsKernelDebuggerPresent	D	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GetMonitorInfo	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GetVolumeInformation	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GetSystemFirmwareTable	V	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		NtQuerySystemInformation$PHYSICAL_MEMORY_INFO)	V	0	◦	•	◦	◦	◦	•	•	◦	◦	◦	◦
		NtQuerySystemInformation(0x23)	D	2	◦	◦	◦	◦	◦	•	◦	◦	◦	◦	◦
		DeviceIoControl	V	6	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		PEB->NumProcessors	V	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		NtQueryObject after NtCreateDebugObject	D	0	•	◦	◦	◦	◦	◦	◦	•	•	•	•
		FindWindow	M	3	◦	◦	◦	◦	◦	◦	◦	◦	•	◦	◦
		GetLocaleInfo	V	2	•	•	•	•	•	•	•	•	•	•	•
		GetComputerName	V	1	•	•	•	•	•	•	•	◦	◦	◦	◦
		GetUserDefaultUILanguage	V	1	•	•	•	•	•	•	•	•	•	•	•
		GetSystemDefaultUILanguage	V	0	•	•	•	•	•	•	•	•	•	•	•
		IsDebuggerPresent	D	9	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		GetAdaptersInfo	V	2	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GetDiskFreeSpace/Ex	V	0	•	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		CheckRemoteDebuggerPresent	D	3	◦	◦	◦	◦	◦	◦	◦	•	•	•	•
		GetAdaptersAddresses	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		GlobalMemoryStatusEx	V	1	◦	•	◦	•	◦	•	•	◦	◦	◦	◦
Stalling	[30] [69] [56]	**Ping**	M	0	•	•	◦	•	◦	•	◦	◦	◦	◦	◦
	[34] [50]	**Beep**	M	0	•	•	◦	◦	◦	•	◦	◦	◦	◦	◦
		WaitForMultipleObjects	M	0	•	•	•	•	◦	•	◦	◦	◦	◦	◦
		waitForSingleObject/Ex	M	1	•	•	•	•	◦	•	◦	◦	◦	◦	◦
		BeepLoop	M	0	•	•	◦	◦	•	•	◦	◦	◦	◦	◦
		Sleep/SleepEx	M	3	•	•	◦	◦	◦	•	◦	◦	◦	◦	◦
		SetTimer	M	1	•	•	•	•	◦	•	◦	◦	◦	◦	◦
		SetWaitableTimer/Ex	M	0	•	•	•	•	◦	•	◦	◦	◦	◦	◦
		CreateTimerQueueTimer	M	0	•	•	•	•	◦	•	◦	•	◦	◦	◦
		NtDelayExecution	M	1	•	•	◦	◦	◦	•	◦	◦	◦	◦	◦
		timeSetEvent	M	0	•	•	•	•	•	•	•	◦	◦	◦	◦
		icmpSendEcho/2/Ex	M	0	•	•	•	•	◦	•	◦	◦	◦	◦	◦
		SleepLoop	M	0	•	•	◦	•	•	•	◦	◦	◦	◦	◦
Table	[64]	SLDT	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
Descriptors		SGDT	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦

(Continued)

Table 1. Continued

Category	References	Technique	T	P	S1	S2	S3	S4	S5	S6	S7	D1	D2	D3	D4
Timing	[43] [52] [64]	GetTickCount	M	4	•	•	◦	•	◦	◦	•	◦	◦	◦	◦
		timeGetSystemTime	M	0	•	•	◦	•	•	•	•	◦	◦	◦	◦
		KUSER_SHARED->TickCountQuad	M	2	◦	◦	◦	◦	◦	•	•	◦	◦	◦	◦
		NtGetTickCount	M	0	◦	◦	◦	◦	◦	•	•	◦	◦	◦	◦
		QueryInterruptTime	M	0	•	•	◦	•	•	•	•	◦	◦	◦	◦
		NtQueryPerformanceCounter	M	6	•	•	◦	•	•	•	•	•	•	◦	◦
		QueryUnbiasedInterruptTimePrecise	M	0	•	•	•	•	•	•	•	•	•	◦	◦
		QueryInterruptTimePrecise	M	0	•	•	•	•	•	•	•	•	•	◦	◦
		GetTickCount64	M	0	•	•	◦	•	◦	•	•	◦	◦	◦	◦
		QueryUnbiasedInterruptTime	M	0	•	•	◦	•	◦	◦	•	◦	◦	◦	◦
		RDTSC/D	M	2	•	•	•	•	•	•	•	•	◦	◦	◦
		QueryPerformanceCounter	M	9	•	•	•	•	•	•	•	◦	◦	◦	◦
		GetLocalTime	M	1	•	•	◦	•	◦	•	•	◦	◦	◦	◦
		timeGetTime	M	1	◦	•	◦	◦	•	•	•	◦	◦	◦	◦
		NtQuerySystemTime	M	10	•	•	◦	•	•	•	•	◦	•	◦	◦
		GetSystemTime	M	0	•	•	◦	•	◦	•	•	◦	◦	◦	◦
		KUSER_SHARED->SystemTime	M	2	◦	◦	◦	◦	◦	◦	•	◦	◦	◦	◦
		KUSER_SHARED->InterruptTime	M	1	◦	◦	◦	◦	◦	◦	•	◦	◦	◦	◦
Traps	[22] [27]	INT 3	D	0	◦	◦	◦	◦	•	◦	◦	•	•	•	•
	[35] [64]	VPCEXT	V	0	◦	◦	◦	◦	•	◦	◦	•	•	•	•
		INT 1	D	2	◦	◦	◦	◦	•	◦	◦	•	•	•	•
		ICEBP	D	1	◦	◦	◦	◦	•	◦	◦	•	•	•	•
		INT 2D	D	0	◦	◦	◦	◦	•	◦	◦	•	•	•	•
WMI	[32] [56] [42]	**LogicalDisk**	V	0	◦	◦	◦	•	◦	◦	◦	◦	◦	◦	◦
	[63] [39] [30]	**NetworkAdapterConfiguration**	V	0	◦	◦	◦	◦	•	◦	◦	◦	◦	◦	◦
	[44] [23]	**PnPEntity**	V	0	•	•	•	•	•	•	•	•	•	•	•
		MSAcpi_ThermalZoneTemperature	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		PointingDevice	V	0	◦	◦	•	◦	•	◦	◦	◦	◦	◦	◦
		DiskDrive	V	0	◦	◦	•	◦	•	◦	◦	◦	◦	◦	◦
		BaseBoard	V	0	◦	◦	•	◦	◦	◦	◦	◦	◦	◦	◦
		VideoController	V	0	◦	◦	•	◦	•	◦	◦	◦	◦	◦	◦
		NetworkAdapter	V	0	◦	•	◦	◦	•	•	•	◦	◦	•	◦
		OperatingSystem	V	0	◦	•	◦	◦	•	•	◦	◦	◦	•	◦
		Bus	V	0	•	◦	◦	◦	•	◦	◦	◦	◦	◦	◦
		ComputerSystemProduct	V	0	◦	◦	◦	◦	•	◦	◦	◦	◦	◦	◦
		SecurityCenter2_AntiVirusProduct	V	0	•	◦	•	•	◦	•	◦	•	•	•	•
		Processor	V	0	◦	•	◦	◦	◦	•	◦	◦	◦	◦	◦
		MemoryDevice	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		MemoryArray	V	0	◦	◦	•	◦	◦	◦	◦	◦	◦	◦	◦
		VoltageProbe	V	0	◦	◦	•	◦	◦	◦	◦	◦	◦	◦	◦
		PortConnector	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		SMBIOSMemory	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		PerfFormattedData_Counters _ThermalZoneInformation	V	0	◦	◦	•	•	◦	◦	◦	◦	◦	◦	◦
		Memory	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		Sensor	V	0	◦	◦	•	◦	•	◦	◦	◦	◦	◦	◦
		NumericSensor	V	0	◦	◦	•	•	◦	◦	◦	◦	◦	◦	◦
		TemperatureSensor	V	0	◦	◦	•	◦	◦	◦	◦	◦	◦	◦	◦
		VoltageSensor	V	0	◦	◦	•	◦	◦	◦	◦	◦	◦	◦	◦
		PhysicalConnector	V	0	◦	◦	◦	•	•	◦	◦	◦	◦	◦	◦
		Slot	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		BIOS	V	0	◦	◦	◦	◦	•	◦	◦	◦	◦	◦	◦
		ComputerSystem (model)	V	0	◦	◦	•	◦	•	◦	◦	◦	◦	◦	◦
		ComputerSystem (manufacturer)	V	0	◦	◦	◦	•	•	•	◦	◦	◦	◦	◦
		ComputerSystem (ProcessorId)	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		Fan	V	0	◦	◦	•	•	◦	◦	◦	◦	◦	◦	◦
		CacheMemory	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦
		PhysicalMemory	V	0	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦

Experiments shown in bold represent those newly identified in our updated taxonomy. Column T indicates the type of technique, where V stands for anti-virtualization, D for anti-debugging, and M for miscellaneous. Column P shows how many packers use a specific evasive technique. A • indicates that a technique successfully evaded detection in a given environment, while a ◦ indicates that it did not.

— **Memory Fingerprinting** The evasive techniques in this category look for traces of analysis environments in the memory space of the running process. For example, process-related structures like the **Process Environment Block** (**PEB**) or the KUSER_SHARED_DATA contain information about any debugger attached to the process, both in user space and kernel space.
— **CPU Fingerprinting** These techniques use specific assembly instructions, e.g., CPUID, to query information from the CPU running the process, to determine if the processor is a real one or if the program is running through a hypervisor.
— **Table Descriptor**: malware uses the values present inside the Table Descriptors allows to check for abnormal values that are typically present inside virtual machines and sandboxes. However, such techniques work only on single-core machines and are now deprecated. In particular, these techniques search for analysis indicators inside the **Local Descriptor Table** (**LDT**), the **Global Descriptor Table** (**GDT**), and the **Interrupt Descriptor Table** (**IDT**).
— **Traps** These techniques execute some CPU traps or interrupt instructions that may be intercepted by analysis environments, preventing the program from handling them.
— **Registry** Techniques of this category query the system registry of Windows for known keys related to virtual machines, sandboxes or installed debuggers.
— **WMI Windows Management Instrumentation** (**WMI**) is a powerful framework that provides a standardized way to manage and interact with various components of a Windows operating system. Among the offered functionalities is the **WMI Query Language** (**WQL**), which has a SQL-like syntax and allows users to get information about the current Windows installation, such as process lists, installed hardware, and machine information. Techniques in this category use WMI to detect if the process is running inside an analysis environment.
— **Process Environment** This type of check looks into the Process Environment data structure for evidence of any analysis environment, such as process information, any loaded debugging or analysis module, or manipulation of the context of the current thread, since this is an operation typically intercepted by a debugger.
— **File System** Techniques in this category checks for files on disk related to virtualization or analysis tools. Moreover, there are checks about the path of the current process, since many automated sandboxes run samples from folders containing the "sandbox" string.
— **List Process** Enumerates active processes, looking for known debugger or sandbox-related processes.
— **List Services** Similar to drivers, processes, files, and other virtualization-related artifacts, malware may enumerate services to check whether they are linked to virtualization software. This can be accomplished by obtaining the name of each service through methods such as EnumServicesStatus, GetServiceDisplayName, or GetServiceKeyName, and subsequently comparing it against a predefined blacklist, or by invoking OpenService with a blacklisted name.
— **Drivers Information** Virtual Machines often install drivers to improve the interaction between the guest and the host operating system. Some evasive techniques look for virtual machine related drivers, stopping the malware's execution whenever such a driver is found.
— **Other** A collection of varied checks that do not fit into other categories but are effective at detecting analysis environments.

4.2 Execution-Behavior

This class of evasive techniques analyzes the execution behavior of a process to determine whether it is running on a legitimate system or within an analysis environment. Typical strategies include

detecting the absence of human interaction, delaying execution to exceed sandbox timeouts, and raising exceptions to interfere with debugger control flow.

— **Human Interaction** Most advanced evasive techniques perform reverse Turing tests to detect if the process is automatically run inside a sandbox or launched by a real human on a legitimate machine. These techniques check for human activity before exposing the malicious behavior of the malware. If the reverse Turing test fails, e.g., the malware does not detect any human activity, the execution is terminated.
— **Stalling** These techniques perform blocking operations to exceed the monitoring window of dynamic analysis systems. Techniques such as `WaitForMultipleObjects`, `WaitForSingleObject(Ex)`, `timeSetEvent`, `SetTimer`, `SetWaitableTimer`, and `CreateTimerQueueTimer` combine kernel-level waiting with timer- or event-based signaling, making them harder to accelerate or bypass. Additionally, execution can be blocked using loop-based delays (`SleepLoop`, `BeepLoop`) to repeatedly pause, or network/I/O operations (`Ping`, `icmpSendEcho`) to introduce external waiting.
— **Exception Handling** Techniques under this category raise exceptions to test if they are correctly passed to the program. In fact, many analysis tools intercept exceptions by default and does not pass them to the running program.

4.3 Side-Channel Checks

This class of attacks detects computational overhead introduced by the analysis environment. Both sandboxes and debuggers slow down process execution: the former typically run inside virtual machines with constrained resources, while the latter incur delays due to human interaction. Recent studies show that such attacks can significantly increase the evasion rate of malware by employing Proof-of-Work algorithms [48]. Our work shows that most packers introduce side-channel measurements into the output binary and that sandboxes are highly vulnerable to this class of checks.

— **Timing** This family of techniques identifies analysis environments by checking for discrepancies in the expected execution time. We evaluate two representative timing-based checks: the execution overhead of the `CPUID` instruction and the consistency of the `Sleep` function behavior. The `CPUID` instruction executes significantly slower inside a hypervisor, as it typically triggers a VM-exit transition to the host, introducing additional latency compared with execution on bare metal. By measuring the execution time of repeated `CPUID` invocations, a program can infer the presence of virtualization. To define a threshold, we do multiple measurements on a bare-metal machine, and computes a mean of the different execution times. The `Sleep`-based checks verify that the observed suspension time is not shorter than the requested interval. This allows detecting analysis environments that hook or modify sleep-related APIs to avoid dormant behavior. A discrepancy between the expected and observed sleep duration is a sign of being under analysis.

In addition to grouping techniques, we further classify them into three categories: **anti-virtualization**, **anti-debugging**, and **miscellaneous**. **Anti-virtualization** techniques are designed to detect whether the program is running inside a virtualized environment. **Anti-debugging** techniques aim at determining if the program is being executed under a debugger. Finally, **miscellaneous** techniques target generic analysis environments, by performing checks that are effective against both virtual machines and debuggers, e.g., timing analyses revealing execution overhead.

Table 1 presents the techniques included in our taxonomy, along with the results obtained against each analysis environment tested. The P column represents the number of packers adding a specific technique to the obfuscated program, while the T column specify the type of a technique, where

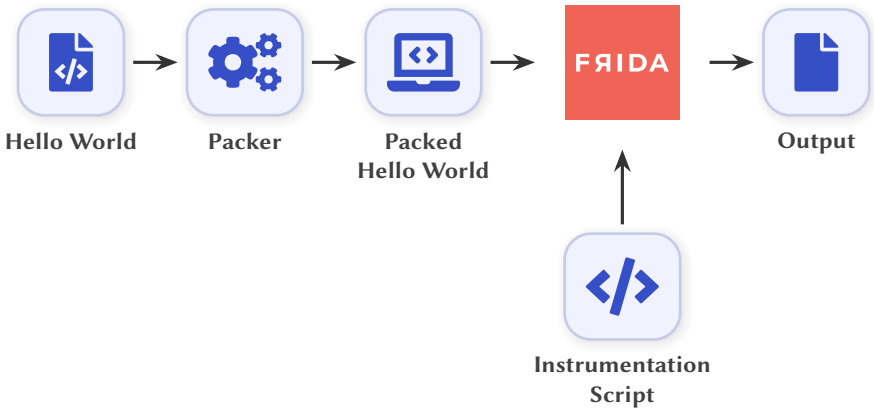


Fig. 1. Sandbox evasion evaluation process.

D stands for anti-debugging, V for anti-virtualization, M for miscellaneous. The other columns correspond to a specific analysis tool: columns prefixed with S represent sandboxes, while those prefixed with D represent debuggers. For each technique–environment pair, we use a • to indicate a successful evasion (i.e., the technique bypassed detection by the tool), a ◦ to indicate a failed evasion (i.e., the tool successfully detected the technique), and a—to denote incompatibility between the technique and the tool. For instance, techniques using the Wudf family of Windows APIs are not compatible with 32-bit systems.

5 Methodology

In this section, we introduce the methodology we used to conduct our study. In particular, we explain how we gathered the evasive techniques introduced in obfuscated binaries by commercial packers. Finally, we present the dataset of binaries used in the evaluation of sandboxes and debuggers, and how we studied their readiness against the identified evasive techniques.

5.1 Packers

We employ a dynamic approach to identify the evasive techniques introduced by packers in the generated obfuscated binaries. Figure 1 depicts the overall identification process.

We compile a baseline program, performing minimal operations, with the objective of not affecting the results of our assessment. This program is packed with each packer under study, and the resulting binaries are dynamically analyzed to identify evasive techniques introduced at runtime. In this phase, we exclude both the synthetic dataset and real-world malware. The objective of this evaluation is to isolate and characterize the evasive techniques introduced by the packer. Using dataset binaries or real-world malware would combine payload-level evasive behavior with packer-introduced mechanisms, preventing reliable attribution and compromising the validity of the analysis.

For the dynamic analysis phase, between the two most widely used options, Intel Pin [40] and Frida [54], we opt for the latter, as the former is widely used in malware research and is often explicitly targeted by evasion techniques. Using Frida, we improve the resilience of our tool against such countermeasures. Instrumentation is performed at both function and instruction granularity, to intercept the execution of evasive checks. At the function level, we track calls to Windows APIs using Frida's *Interceptor* API, which allows us to insert trampolines at the entry or exit points of

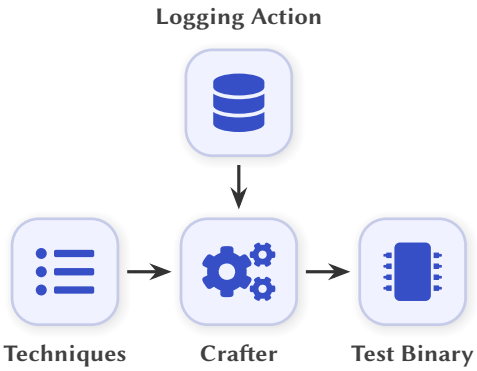


Fig. 2. Synthetic test generation process.

the targeted functions. To counteract evasion, we modify the return values that could reveal the presence of an analysis environment. At the instruction level, we monitor specific x86 instructions commonly used in anti-analysis techniques, such as *CPUID*, which exposes CPU characteristics. Frida's *Stalker* engine facilitates this by using dynamic recompilation: instead of modifying the original code in memory, it generates a rewritten copy in a separate memory region. This allows inserted instrumentation to run while bypassing code integrity checks. As a result, our approach reduces the risk of detection by malware that uses integrity verification to evade analysis.

5.2 Dataset

The dataset we use to validate our approach is synthetically generated and comprises one test binary for each evasive technique identified in our taxonomy, and one test binary per each studied packer.

As illustrated in Figure 2, the generation process consists of wrapping a dedicated *logging action* around a set of evasive techniques. Each test program executes the corresponding evasive checks and, mimicking the behavior of real malware, conditionally performs the logging action, which enables us to record the outcome of the test. This action varies across the evaluated environments (e.g., sandboxes and debuggers) and is determined by the output of the tool under test, as explained in Section 6.2. We conduct a twofold evaluation. First, we evaluate evasive techniques individually, which allows us to assess the readiness of an analysis environment against each specific technique. In this phase, individual assessment is necessary to prevent confounding side effects that could skew the results of a multi-technique analysis.

Then, for each packer under analysis, we generate one binary containing all the techniques implemented by that specific packer. This allows us to assess the readiness of analysis environments against a combination of multiple evasive techniques as used by commercial packers. We deliberately avoid using real malware samples, as they pose two major challenges to experimental validity. First, establishing a reliable ground truth would require prior knowledge of both the evasive mechanisms and the malicious actions implemented by each sample. Second, if the samples are already known, they are typically blocked by the static filtering stages of modern sandboxes, preventing dynamic execution altogether. Our preliminary experiments with real malware confirm this limitation. Finally, since our goal is to assess the robustness of analysis environments against specific evasive techniques—rather than their ability to detect malware—the use of synthetic binaries does not affect the validity of our results: the evasive mechanisms under evaluation remain identical, regardless of whether they are embedded in real or synthetic programs.

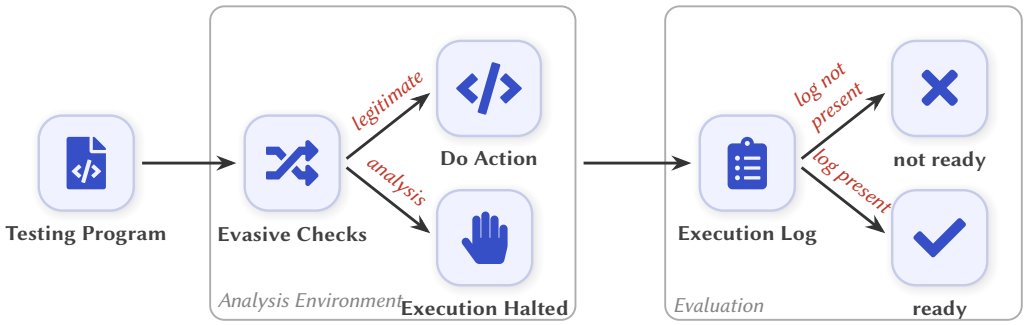


Fig. 3. Graphical representation of our evaluation process: the evasion technique is added to a program and executed in the testing environment. If the program fails to detect the execution environment, it performs a specific action. If this action is later observed in the execution log, we consider the environment capable of handling the evasion technique under test.

5.3 Sandboxes and Debuggers

Figure 3 illustrates the test approach used in analysis environments. A sandbox typically logs various details during the analysis of a sample, including its network activity and the tree of executed processes. Since these logs are returned as part of the analysis output, our test binaries perform actions observable from the output log to help identify whether an evasion attempt has failed. If the sandbox provides network activity logs, the test binary opens a connection to a public server on TCP port 443 when an evasion attempt fails. If this connection appears in the output, we register the evasion attempt as failed; otherwise, we consider it successful. If network activity is not available in the output, the binary instead executes the `dir C:` command using the C `system` function. If this command appears in the recorded process tree, we again mark the evasion attempt as failed. All the sandboxes we tested provided at least one of these logs (network or process), and we manually verified that both network connections and command executions were reliably captured during the analysis.

Unlike sandboxes, debuggers do not produce execution logs by default. However, since our test programs are benign, we run the experiments locally and allow each program to create a file on disk if the evasion attempt fails. If the file is present after execution, we register the evasion attempt as failed; otherwise, we consider it successful. We present the results in two separate sections: one for debuggers (Section 6.3) and one for sandboxes (Section 6.2). For each category of analysis tool, we distinctly evaluate the readiness to counteract anti-virtualization, anti-debugging, and miscellaneous techniques.

To provide a quantitative assessment of how well the tested environments handle the identified evasive techniques, we estimate a readiness probability. This value represents the likelihood that a randomly selected sandbox from those tested is ready to counteract a random technique from a given family. We define *readiness probability (RP)* by the following formula:

$$RP = \frac{\text{evasions}_{fail}}{num_{exp} \cdot num_{tools}}$$

where $evasions_{fail}$ is the total number of unsuccessful evasions observed within that family, num_{exp} is the total number of experiments in the family, and num_{tool} is the total number of sandboxes or debuggers tested.

Table 2. List of Tested Packers with Versions

Packer	Version	Trial	Packer	Version	Trial	Packer	Version	Trial
Alienyze	1.4	Y	PCGuard	6.00.0928	N	Obsidium	1.8.1.2	Y
Alternate	2.510	N	PECompact	3.03	N	Mew11	1.2	N
ASPack	2.43	Y	PELock	2.11	Y	Mpress	2.19	N
ASProtect	2.78	Y	Petite	2.4	N	NSPack	3.7	N
Bero	1.00	N	RLPack	1.17	N	Themida	3.1.3.0	N
EnigmaProtector	7.40	Y	Telock	0.99	N	UPX	4.2.1	N
EnigmaVM	10.50	N	VMProtect	3.8.5	Y	YodaProtector	1.03	N
Exe32	1.42	N	ExeStealth	4.21	N	HXOR	0.1	N
Kkrunchy	0.23	N						

A **Y** in the Trial column indicates the trial version was used.

6 Experimental Results

In this section, we analyze the experimental results obtained against packers, sandboxes, and debuggers. First, we report the evasive techniques introduced by packers during the obfuscation process. Then, we assess the preparedness of sandboxes and debuggers against the techniques identified in our taxonomy, evaluating whether the techniques employed by packers are effective. Finally, we demonstrate the effectiveness of the evasive techniques employed by packers against the tested debuggers and sandboxes Even if we disclose the names of the sandboxes and debuggers evaluated, we do not associate our results with any specific tool. Our goal is to provide a general overview of the current state of readiness of analysis tools against evasive techniques, rather than to rank or single out specific solutions. Similarly, even if we disclose the names of the packers used in our testing, we do not associate specific evasive techniques with individual packers. Our goal is not to rank them based on their ability to evade sandboxes, but rather to assess the overall presence of evasive techniques introduced by packers.

6.1 Evaluation of Packers Evasion Capabilities

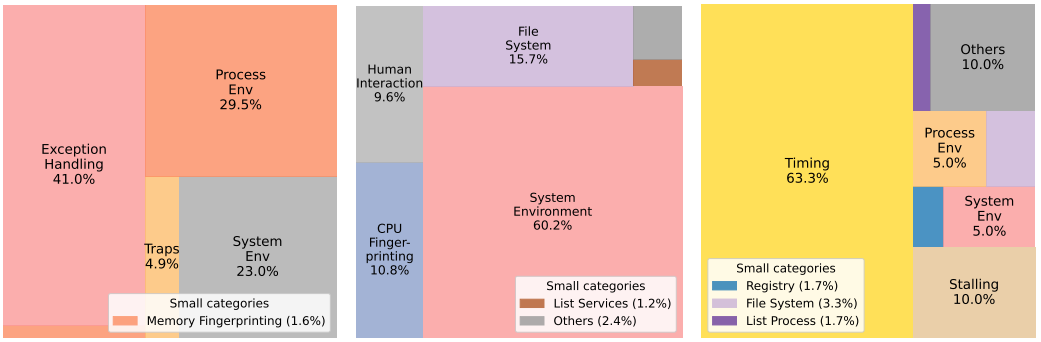
In this section, we analyze the evasive technique employed by packers, answering to:

RQ2: What are the evasive techniques added to obfuscated binaries by available commercial packers in the resulting obfuscated binaries?

Table 2 reports the packers evaluated in our experiments, together with their versions and whether the tests were conducted using trial licenses.

6.1.1 Anti-Debugging. Figure 4(a) illustrates the distribution of categories of anti-debugging technique introduced by the tested packers during obfuscation. The results indicate that most packers primarily rely on techniques related to Exception Handling, System Environment, and Process Environment to detect the presence of a debugger. Table 3 provides detailed statistics on the usage of anti-debugging techniques, reporting for each technique the number of packers that implement it, as well as the number of sandboxes and packers against which it is effective.

6.1.2 Anti-Virtualization. Figure 4(b) illustrates the category distribution of anti-virtualization techniques employed by the tested packers. The results indicate that most packers attempt to detect virtualized environments using techniques from the System Environment, File System, and CPU Fingerprinting categories. Additionally, some packers make limited use of Reverse Turing Test techniques. Table 4 summarizes the anti-virtualization techniques introduced by packers and the readiness ratio of each analysis tool. The aggregated data shows that few packers use anti-virtualization techniques, with at most eight for the most frequently used techniques.



(a) Anti-debugging (b) Anti-virtualization (c) Miscellaneous

Fig. 4. Category distribution of evasive techniques introduced by packers by typology.

Table 3. Summary of Anti-Debugging Techniques Used by Packers

Technique Name	Category	Packers	Sandboxes	Debuggers
SetUnhandledExceptionFilter	Exception Handling	P1, P3, P6, P9, P11, P12 P13, P14, P15, P16, P17 P19, P20, P22, P23, P24	6/7	0/4
IsDebuggerPresent	System Environment	P1, P4, P5, P7, P8 P10, P18, P21, P23	7/7	0/4
POPF/D - TRAP FLAG	Exception Handling	P2, P4, P5, P7, P8, P10 P13, P20, P23	6/7	2/4
NtQueryInformationProcess(0x07)	Process Environment	P5, P8, P13, P18, P21 P23	7/7	0/4
NtSetInformationThread(0x11)	Process Environment	P8, P13, P21, P23	0/7	0/4
DebugActiveProcess on Parent	Process Environment	P7, P8, P16	7/7	4/4
NtQueryInformationProcess(0x1e)	Process Environment	P13, P21, P23	7/7	0/4
CheckRemoteDebuggerPresent	System Environment	P5, P8, P23	7/7	0/4
NtQueryInformationProcess(0x1f)	Process Environment	P8, P21	6/7	0/4
NtQuerySystemInformation(0x23)	System Environment	P13, P21	6/7	4/4
INT 1	Traps	P2, P7	6/7	0/4
ICEBP	Traps	P2	6/7	0/4
OutputDebugString	Exception Handling	P21	7/7	0/4
NtClose(INVALID_HANDLE)	Exception Handling	P13	7/7	2/4
PEB->BeingDebugged	Memory Fingerprinting	P13	7/7	0/4

For each technique, the table reports its category, the number of packers that implement it, and the readiness of the analysis tools against it—expressed as the number of sandboxes and debuggers that successfully counter the evasion (e.g., 3/7 indicates that 3 out of 7 sandboxes neutralized the technique).

6.1.3 Miscellaneous. Figure 4(c) presents the distribution of miscellaneous techniques used by packers, highlighting the tendency to measure execution overhead as a means of detecting analysis environments. Specifically, Timing techniques are the most prevalent, accounting for 63.3% of the total. Table 5 reports the number of packers adopting each technique, further confirming that packers predominantly rely on timing-based methods in this category.

The analysis reveals that the most commonly introduced evasive techniques are from the Exception Handling, System Environment, and Process Environment categories. These are predominantly used for anti-debugging and anti-virtualization purposes. Specifically, techniques like SetUnhandledExceptionFilter, IsDebuggerPresent, and NtQueryInformationProcess were the most frequently adopted anti-debugging methods. On the anti-virtualization front, commonly

Table 4. Summary of Anti-Virtualization Techniques Used by Packers

Technique	Category	Packers	Sandboxes	Debuggers
GetModuleFileNameA	File System	P1, P5, P8, P10, P13 P20, P21, P23	6/7	0/4
GetSystemMetrics	System Environment	P1, P4, P5, P8, P10 P18, P21, P23	5/7	4/4
CPUID (EAX=0x00000001)	CPU Fingerprinting	P4, P5, P13, P14, P19, P20, P21, P24	4/7	4/4
GetDesktopWindowRect	System Environment	P4, P5, P8, P10, P21, P23	5/7	4/4
DeviceIoControl	System Environment	P1, P7, P8, P10, P20, P21	7/7	4/4
GetProcessImageFileNameA	File System	P4, P5, P10, P21, P23	6/7	0/4
GetKeyboardLayout	System Environment	P4, P5, P8, P10, P21	0/7	0/4
GetKeyboardLayoutList	System Environment	P4, P5, P8, P10, P21	0/7	0/4
GetForegroundWindow	Human Interaction	P4, P5, P10, P21, P23	4/7	4/4
GetUserDefaultLangID	System Environment	P4, P5, P10, P21	0/7	0/4
GetLogicalDriveType	Others	P8, P20	7/7	4/4
GetLocaleInfo	System Environment	P8, P18	0/7	0/4
GetProcessAffinityMask	System Environment	P13, P18	7/7	4/4
GetSystemInfo	System Environment	P13, P18	7/7	4/4
GetAdaptersInfo	System Environment	P10, P13	7/7	4/4
GetCursorPos	Human Interaction	P1, P8	4/7	4/4
OpenSCManager and EnumServicesStatus	List Services	P8	6/7	4/4
GetSystemFirmwareTable	System Environment	P23	7/7	4/4
GetUserDefaultUILanguage	System Environment	P23	7/7	0/4
EnumDisplayMonitors	System Environment	P8	7/7	4/4
EnumWindows	System Environment	P8	7/7	4/4
GetUsernameA	System Environment	P8	7/7	4/4
PEB->NumProcessors	System Environment	P4	7/7	4/4
GlobalMemoryStatusEx	System Environment	P23	3/7	4/4
GetComputerName	System Environment	P8	0/7	4/4
SetWindowsHookExA	Human Interaction	P8	1/7	4/4
CPUID (EAX=0x40000000)	CPU Fingerprinting	P13	4/7	4/4

For each technique, the table reports its category, the number of packers that implement it, and the readiness of the analysis tools against it—expressed as the number of sandboxes and debuggers that successfully counter the evasion (e.g., 3/7 indicates that 3 out of 7 sandboxes neutralized the technique)

used techniques included GetModuleFileNameA, GetSystemMetrics, and CPUID. Additionally, the Timing category emerged as the most prevalent in miscellaneous evasions, with techniques such as NtQuerySystemTime, QueryPerformanceCounter, and NtQueryPerformanceCounter being widely implemented to detect emulated or slowed-down environments.

6.2 Evaluation of Sandboxes' Handling of Evasion Techniques

The results of the experiments we conducted against sandboxes allow us to answer to

RQ3: What is the coverage of publicly available analysis environments (i.e., sandboxes and debuggers) in handling evasive techniques?

RQ4: What is the effectiveness of techniques added by packers against the tested analysis environments?

We conducted our experiments in 7 publicly available sandboxes, chosen as a subset of the most famous available sandboxes [79]. In particular, those sandboxes are, in no particular order, **Triage** [70], **Intezer** [28], **Hybrid Analysis**, **Pikker** [51], **C2AE** [10] (through VirusTotal), **Qianxin** [53],

Table 5. Summary of Miscellaneous Techniques Used by Packers

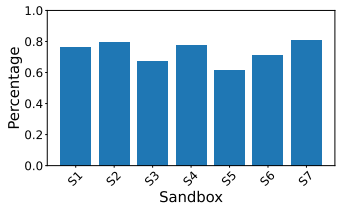
Technique	Category	Packers	Sandboxes	Debuggers
NtQuerySystemTime	Timing	P4, P11, P13, P14, P15 P16, P19, P21, P22, P23	0/7	0/4
QueryPerformanceCounter	Timing	P8, P11, P13, P14, P15 P16, P21, P22, P23	0/7	4/4
NtQueryPerformanceCounter	Timing	P1, P4, P5, P17, P18, P21	0/7	3/4
CanOpenCsrss	Others	P5, P8, P10, P18, P21	5/7	4/4
GetTickCount	Timing	P8, P10, P13, P16	3/7	4/4
GetModuleHandle	Process Environment	P7, P13, P16	6/7	4/4
FindWindow	System Environment	P8, P21, P23	7/7	3/4
Sleep/SleepEx	Stalling	P8, P20, P23	4/7	4/4
KUSER_SHARED->TickCountQuad	Timing	P4, P8	5/7	4/4
KUSER_SHARED->SystemTime	Timing	P4, P8	6/7	4/4
RDTSC/D	Timing	P13, P23	0/7	4/4
IsHooked	Others	P7	7/7	4/4
EnumProcesses and GetModuleBaseName	List Process	P8	6/7	4/4
NtCreateFile	File System	P8	4/7	4/4
NtOpenFile	File System	P13	6/7	4/4
NtOpenKey/Ex	Registry	P13	3/7	0/4
NtDelayExecution	Stalling	P13	4/7	4/4
SetTimer	Stalling	P8	2/7	4/4
waitForSingleObject/Ex	Stalling	P19	2/7	4/4
KUSER_SHARED->InterruptTime	Timing	P4	6/7	4/4
timeGetTime	Timing	P23	3/7	4/4
GetLocalTime	Timing	P8	0/7	1/4

For each technique, the table reports its category, the number of packers that implement it, and the readiness of the analysis tools against it—expressed as the number of sandboxes and debuggers that successfully counter the evasion (e.g., 3/7 indicates that 3 out of 7 sandboxes neutralized the technique).

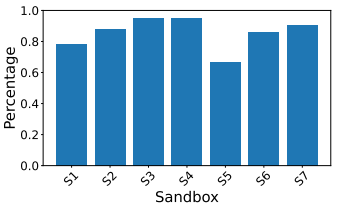
Any.run [4]. Table 1 contains the full results of the experiments, showing for each pair (t, s) of techniques and sandboxes if t evaded execution on s. Since all tested sandboxes support 64-bit executables, we test with 64-bit binaries, since most Windows APIs are not available in 32-bit compatibility mode.

6.2.1 Anti-Debugging. The results shown in Figure 5(a) and Table 6 highlight the high readiness of online sandboxes against anti-debugging techniques, with an average of 73% of such techniques successfully countered by the tested sandboxes. We report in Table 6 the techniques with the highest evasion ratio. `NtSetInformationThread(0x11)` stands out as the most evasive technique, successfully evading in all the tested sandboxes. However, even this technique represents an outlier rather than a consistent trend, as sandboxes can circumvent evasion attempts of most techniques.

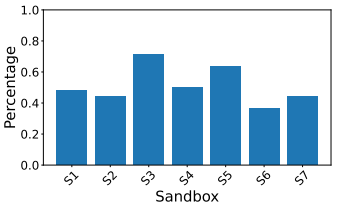
Table 7 presents a detailed overview of sandbox readiness against various categories of anti-debugging techniques. Techniques in the List Process family exhibit the lowest readiness probability (71.43%), indicating notable weaknesses in how sandboxes handle queries related to process enumeration. Similarly, techniques from the Process Environment category show moderate readiness (76.79%), highlighting ongoing difficulties in fully concealing debugger artifacts within a process's internal structure. Conversely, sandboxes demonstrate strong resilience against the System Environment and Memory Fingerprinting categories, with high readiness probabilities of 97.14% and 94.29%, respectively. Notably, the high readiness probabilities observed for the Exception Handling (88.10%) and Traps (85.71%) categories suggest that most sandboxes manage signals and exceptions transparently, typically passing them directly to the analyzed program without interference. These results, together with those presented in Table 3, indicate a high level of



(a) Anti-debugging



(b) Anti-virtualization



(c) Miscellaneous

Fig. 5. Readiness of tested sandboxes against the evasive techniques in our taxonomy. Each bar represents the percentage of techniques that a sandbox successfully countered, relative to the total number of applicable techniques.

Table 6. Most Evasive Anti-Debugging Techniques

Experiment Name	Category	Count
NtSetInformationThread(0x11)	Process Environment	7
NtYieldExecution	Others	5
NtSetContextThread(CONTEXT_DEBUG_REGISTERS)	Process Environment	3
POPF/D - TRAP FLAG	Exception Handling	2
INT 1	Traps	2
NtCreateThreadEx(HIDE_FROM_DEBUGGER)	Process Environment	2
CTRL Exception Handling	Exception Handling	2
GetProcessIdFromName	List Process	2
DebugBreak	Exception Handling	1
SetUnhandledExceptionFilter	Exception Handling	1

The table lists the anti-debugging techniques that most frequently bypassed detection by the tested sandboxes. For each technique, the table shows its category and the number of sandboxes it successfully evaded.

resilience among sandboxes against anti-debugging techniques employed by packers: most techniques fail to successfully evade detection. The only technique capable of evading all tested sandboxes, *NtSetInformationThread(0x11)*, is implemented by only 4 of the 25 evaluated packers. On average, each anti-debugging technique is effectively neutralized by nearly all sandboxes, affecting just 0.57 sandboxes per technique—highlighting the robust defensive capabilities of these analysis environments.

6.2.2 Anti-Virtualization. Figure 5(b) shows the percentage of anti-virtualization techniques successfully countered by each tested sandbox. On average, the sandboxes demonstrate a readiness of 86%, indicating strong capabilities in concealing artifacts that would reveal the presence of a virtualized environment.

Table 7. Readiness Probability of Sandboxes Against Anti-Debugging Techniques, Grouped by Category

Category	Success	Failed	Techniques	RP
System Environment	68	2	10	97.14%
Memory Fingerprinting	33	2	5	94.29%
Exception Handling	37	5	6	88.10%
Traps	24	4	5	85.71%
Others	42	7	7	85.71%
Process Environment	43	13	8	76.79%
List Process	5	2	1	71.43%

The *Success* column indicates the total number of successfully countered evasion attempts, the *Failed* column indicates the total number of unsuccessfully countered evasion attempts, while *Techniques* represents the total number of techniques evaluated in that category. The *RP* column shows the resulting Readiness Probability.

Table 8. Readiness Probability of Debuggers Against Anti-Virtualization Techniques, Grouped by Category

Category	Success	Failed	Techniques	RP
Table Descr.	21	0	3	100.00%
List Services	26	2	4	92.86%
Emulation softwares	26	2	4	92.86%
Others	37	5	6	88.10%
Traps	6	1	1	85.71%
File System	17	3	3	85.00%
WMI	186	52	34	78.15%
Drivers Information	5	2	1	71.43%
System Environment	139	71	30	66.19%
CPU Fingerprinting	20	15	5	60.00%
Human Interaction	19	30	7	38.78%

The *Success* column indicates the total number of successfully countered evasion attempts, the *Failed* column indicates the total number of unsuccessfully countered evasion attempts, while *Techniques* represents the total number of techniques evaluated in that category. The *RP* column shows the resulting Readiness Probability.

Table 8 presents the readiness probabilities of sandboxes against anti-virtualization techniques, grouped by family. While the overall readiness is relatively high, our experiments reveal a low capacity for circumventing Reverse Turing Tests and in hiding virtualized CPU information. The low readiness against Reverse Turing Tests reveals the inability of sandboxes to simulate human activity, e.g., mouse scrolling, mouse clicks, or keyboard input. As Table 1 shows, it is possible to detect a sandbox by just waiting for mouse or keyboard input. In addition, several sandboxes exhibit poor performance in hiding low-level virtualization artifacts. For example, the SMSW technique, which checks for the presence of a real CPU, successfully evaded in all sandboxes tested. CPUID-based techniques also proved effective: three sandboxes exposed virtualization details when queried with EAX = 0x1 (e.g., number of cores), and other three when queried with EAX = 0x40000000 (e.g., hypervisor information). The weaknesses extend beyond CPU fingerprinting into the broader hardware environment. Techniques based on WMI, which query system properties such as serial numbers and device IDs, often return values indicative of virtualized hardware—highlighting the low readiness of several sandboxes in concealing hardware-based virtualization artifacts. Although

Table 9. Readiness Probability of Debuggers Against Miscellaneous Techniques, Grouped by Category

Category	Successful	Evaded	Techniques	RP
Process Environment	7	0	1	100.00%
System Environment	7	0	1	100.00%
List Process	13	1	2	92.86%
Others	42	7	7	85.71%
Registry	19	9	4	67.86%
File System	25	15	6	62.50%
Stalling	32	59	13	35.16%
Timing	41	85	18	32.54%

The *Success* column indicates the total number of successfully countered evasion attempts, the *Failed* column indicates the total number of unsuccessfully countered evasion attempts, while *Techniques* represents the total number of techniques evaluated in that category. The *RP* column shows the resulting Readiness Probability.

techniques from the System Environment family generally saw higher readiness, all locale-based checks successfully bypassed detection. This suggests that many sandboxes are configured with uncommon system locales, which can serve as a reliable indicator of an analysis environment. Table 4 presents the readiness rates of sandboxes against anti-virtualization techniques introduced by packers. Overall, the results indicate a high level of readiness, as the most commonly used techniques are successfully handled by an average of five sandboxes.

6.2.3 Miscellaneous. Figure 5(c) shows the percentage of miscellaneous techniques handled by each tested sandbox. On average, each sandbox handles 42% of these techniques, showing that sandboxes are more vulnerable to techniques identifying a generic execution environment, rather than a specific anti-virtualization or anti-debugging artifact. Table 9 presents the readiness probabilities of sandboxes against each family of miscellaneous evasive techniques. The results show particularly low readiness for Timing and Stalling techniques, which consistently successfully evade. This highlights a key weakness in the ability of sandboxes to conceal delays introduced by virtualization overhead or analysis hooks, making them easily identifiable through timing-based heuristics. The Registry family also reveals areas of limited readiness. Many techniques in this category successfully retrieve registry keys linked to analysis artifacts, indicating that the tested sandboxes do not take adequate steps to hide or sanitize registry information. Similarly, Table 5 confirms the low readiness of sandboxes against miscellaneous techniques implemented by packers. The most used techniques fall within the Timing category, and all of them were able to evade detection across all tested environments, emphasizing the persistent challenge timing-based methods pose to modern sandbox defenses.

6.2.4 Combination of Packers' Evasive Techniques. Table 10 reports the results of testing combinations of techniques from the analyzed packers against the tested sandboxes. Sandboxes S2 and S4 were excluded from this evaluation as it was conducted during a period when we no longer had access to those specific environments.

The results show that sandboxes are not ready to handle the combination of techniques employed by commercial packers. The most resilient sandbox withstands fewer than half of the tested packers (44%), while the least resilient withstands only 4% of the packers' techniques.

Answer to RQ3 and RQ4: Sandboxes demonstrate strong resilience against *anti-debugging*, with an average of 72% countered techniques, effectively neutralizing the vast majority of them.

Table 10. Overview of Readiness of Sandboxes Against Combination of Techniques Employed by Commercial Packers

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11	PC12	PC13	PC14	PC15	PC16	PC17	PC18	PC19	PC20	PC21	PC22	PC23	PC24	PC25
S1	•	◦	◦	•	•	◦	◦	•	◦	•	•	◦	•	•	•	•	•	•	•	•	•	•	•	•	◦
S3	◦	◦	◦	•	•	◦	◦	•	◦	•	•	◦	•	•	•	•	◦	•	•	◦	•	•	•	◦	◦
S5	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	◦
S6	•	◦	◦	•	•	◦	◦	•	◦	•	•	◦	•	•	•	•	•	•	•	•	•	•	•	◦	◦
S7	•	◦	◦	•	•	◦	◦	•	◦	•	•	◦	•	•	•	•	•	•	•	◦	•	•	•	◦	◦

Each column represent the combination of techniques employed by a specific packer, while rows correspond to a sandbox. A • indicates that the combination of techniques PC successfully evaded the analysis, while a ◦ indicates that they did not.

Although some weaknesses remain in handling process enumeration and internal environment queries, these techniques are rarely employed by packers, reinforcing the robustness of sandbox defenses in this category. Similarly, they are generally effective at countering *anti-virtualization* techniques, with an average readiness of 85%. However, they remain vulnerable to Reverse Turing Test, CPU fingerprinting, and locale-based checks, highlighting persistent limitations in simulating user behavior and concealing low-level virtualization artifacts. In contrast, when evasive techniques are combined as implemented by commercial packers, sandbox readiness drops substantially. Our results show that no sandbox is resilient to more than 44% of the tested packers, while the least resilient withstands only 4% of them. This highlights a significant gap between isolated technique handling and real-world evasion scenarios, indicating that current sandbox defenses remain insufficient against the composite evasion strategies employed in practice. Finally, sandboxes exhibit low readiness against *miscellaneous* evasive techniques, with an average of 42% handled techniques, particularly those based on timing and stalling. These categories remain a persistent challenge due to the inherent overhead of sandbox environments, and they are frequently exploited by packers to evade detection.

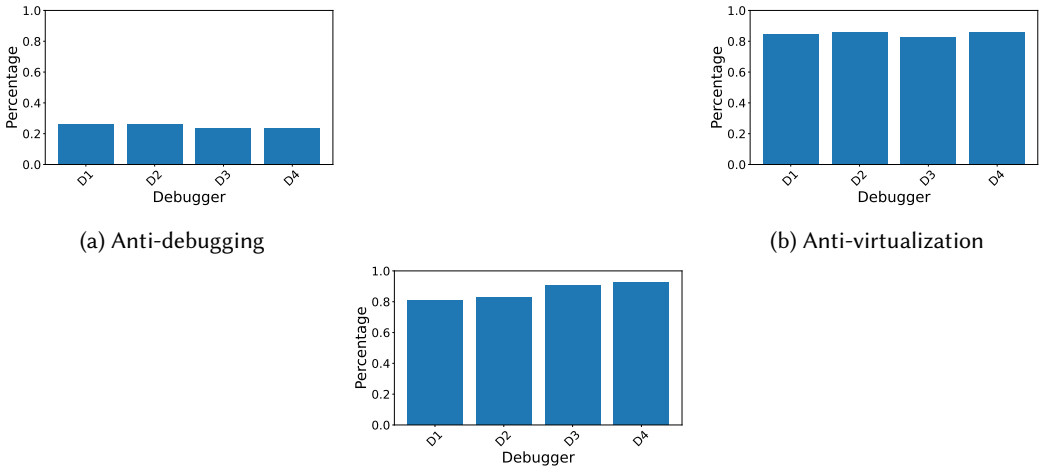
6.3 Evaluation of Debuggers' Handling of Evasion Techniques

As for sandboxes, the analysis of experiments conducted against debuggers allow us to answer to

RQ3: What is the coverage of publicly available analysis environments (i.e., sandboxes and debuggers) in handling evasive techniques?

RQ4: What is the effectiveness of techniques added by packers against the tested analysis environments?

We tested four of the most famous debuggers available for the Windows platform, namely, **WinDbg**, **x64dbg**, **OllyDbg**, **Immunity Debugger**. Two of them, debuggers 3 and 4, support 64-bit executables, while the remaining two, debuggers 1 and 2, only support 32-bit executables, and do not provide a version supporting 64-bits executables. This architectural distinction introduces inherent discrepancies in the results, as some techniques rely on system APIs which are available only on a specific architecure. For instance, the *Wudf* family of APIs was introduces with the 64-bit version of Windows kernel and can therefore only be invoked from 64-bit executables. Aside from these cases, the majority of the evasive techniques considered in this work rely on OS-level interfaces and analysis-environment artifacts that are largely architecture-independent, as they primarily employ standard Windows APIs and process environment properties. We therefore expect the qualitative conclusions to generalize across bitness, while acknowledging that certain techniques are inherently restricted to a specific execution mode. While reporting the results in Table 1, we classify unavailable techniques as successful evasion attempts, since the corresponding evasive checks cannot be executed and, by construction of our synthetic dataset, the malware does not exhibit any malicious behavior.



(a) Anti-debugging

(b) Anti-virtualization

(c) Miscellaneous

Fig. 6. Readiness of tested debuggers against the evasive techniques in our taxonomy. Each bar represents the percentage of techniques that a debugger successfully countered, relative to the total number of applicable techniques.

Table 11. Readiness Probability of Debuggers Against Anti-Debugging Techniques, Grouped by Category

Category	Success	Failed	Techniques	RP
System Environment	18	16	10	52.94%
Others	8	20	7	28.57%
Process Environment	8	24	8	25.00%
Memory Fingerprinting	4	16	5	20.00%
Traps	2	16	5	11.11%
Exception Handling	2	20	6	9.09%
List Process	0	4	1	0.00%

The *Success* column indicates the total number of successfully countered evasion attempts, the *Failed* column indicates the total number of unsuccessfully countered evasion attempts, while *Techniques* represents the total number of techniques evaluated in that category. The *RP* column shows the resulting Readiness Probability.

All tests were performed on a freshly installed version of the Windows 10 operating system, with Virtualization-Based Security disabled to prevent false positives when executing techniques designed to detect the presence of a VM.

6.3.1 Anti-Debugging. Figure 6(a) shows the readiness of the tested debuggers against anti-debugging techniques, revealing that each debugger is ready to counteract evasion on average the 25% of techniques, showing a high tendency to transparently execute the debugged program, without hiding any debugging artifacts. Table 11 presents the readiness probabilities of debuggers against anti-debugging techniques, grouped by category. The results show particularly low readiness for the Exception Handling and Traps families, of just 9.09% and 11.11%, respectively. This suggests that most debuggers, in their default configuration, tend to intercept exceptions and privileged instructions rather than allowing them to pass through transparently—making them easily detectable.

Table 12. Readiness Probability of Debuggers Against Anti-Virtualization Techniques, Grouped by Category

Category	Success	Failed	Techniques	RP
Drivers Information	4	0	1	100.00%
Emulation softwares	16	0	4	100.00%
Human Interaction	28	0	7	100.00%
Table Descr.	12	0	3	100.00%
List Services	16	0	4	100.00%
WMI	126	10	34	92.65%
CPU Fingerprinting	18	2	5	90.00%
Others	20	4	6	83.33%
System Environment	92	28	30	76.67%
File System	0	12	3	0.00%
Traps	0	4	1	0.00%

The *Success* column indicates the total number of successfully countered evasion attempts, the *Failed* column indicates the total number of unsuccessfully countered evasion attempts, while *Techniques* represents the total number of techniques evaluated in that category. The *RP* column shows the resulting Readiness Probability.

Similarly, the Memory Fingerprinting category exhibits a readiness probability of only 20.00%, indicating that the majority of debuggers do not effectively hide debugging artifacts in structures such as the PEB or KUSER_SHARED_DATA. Techniques from the Process Environment family, which typically involve querying the OS for process-specific debugging indicators, were also frequently successful, with debuggers exhibiting a readiness of just 25.00%.

In contrast, the System Environment category showed the highest readiness among all categories, at 52.94%, suggesting a moderate effort in hiding system-wide analysis indicators. However, overall, the data underscores that debuggers remain significantly vulnerable to a broad range of anti-debugging strategies.

Table 3 shows the evasion rate of antidebugging techniques implemented by the packers we tested. In contrast to sandboxes, these techniques demonstrate high effectiveness against debuggers. Most of the anti-debugging methods evaluated successfully target all the debuggers tested, with only a few exceptions. This underscores the significant disparity in the effectiveness of these techniques: while they fail to consistently bypass sandbox environments, they are highly successful in detecting and thwarting debuggers.

6.3.2 Anti-Virtualization. Figure 5(b) illustrates the percentage of anti-virtualization techniques successfully handled by the tested debuggers. The results demonstrate that the debuggers are able to address 80% of these techniques effectively. Table 12 presents the readiness probabilities of debuggers against anti-virtualization techniques, grouped by category. Overall, the results indicate that debuggers are generally ready to counter most categories of evasive techniques. However, lower readiness was observed in the File System, System Environment, and Traps categories.

Several techniques in the File System category, successfully evaded by inspecting the executable path—since the path contained the substring sandbox, or by querying the recent file list, which included entries related to the debugger itself. Similarly, in the System Environment category, locale-based checks consistently bypassed detection due to the experimental setup being configured with an Italian system locale, which is uncommon in real-world targets.

As for the Traps category, the reduced readiness stems from the use of the VPCEXT instruction, which is designed to detect virtual machines. In this case, the debugger's behavior—intercepting the instruction—inadvertently triggered the evasion logic, leading to false positives.

Table 13. Readiness Probability of Debuggers Against Miscellaneous Techniques, Grouped by Category

Category	Success	Failed	Techniques	RP
List Process	8	0	2	100.00%
Process Environment	4	0	1	100.00%
Stalling	51	1	13	98.08%
Timing	60	12	18	88.89%
File System	20	4	6	83.33%
System Environment	3	1	1	75.00%
Registry	12	4	4	75.00%
Others	18	10	7	64.29%

The *Success* column indicates the total number of successfully countered evasion attempts, the *Failed* column indicates the total number of unsuccessfully countered evasion attempts, while *Techniques* represents the total number of techniques evaluated in that category. The *RP* column shows the resulting Readiness Probability.

Table 14. Overview of Readiness of Sandboxes Against Combination of Techniques Employed by Commercial Packers

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11	PC12	PC13	PC14	PC15	PC16	PC17	PC18	PC19	PC20	PC21	PC22	PC23	PC24	PC25
D1	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	◦
D2	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	◦
D3	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	◦
D4	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	◦

Each column represent the combination of techniques employed by a specific packer, while rows correspond to a debugger. A • indicates that the combination of techniques PC, employed by a specific packer, successfully evaded the analysis, while a ◦ indicates that they did not.

Finally, this shows that debuggers are ready to counteract techniques introduces by packers, as reported in Table 4. The few techniques that remain effective against debuggers are primarily based on locale checks and path-based analysis targeting sandbox-related string identification.

6.3.3 Miscellaneous. Figure 6(c) shows the percentage of miscellaneous techniques handled by each tested debugger, revealing that 84.6% of these techniques were neutralized. In other words, only 15.4% of the evaluated strategies were able to evade detection.

Table 13 presents the readiness probabilities of debuggers against various categories of miscellaneous evasive techniques. The Registry category exhibited the lowest readiness (75.00%), primarily due to a single technique that successfully queried registry keys associated with analysis environments. In contrast, Timing and Stalling techniques showed high readiness, with success rates 88.89% and 98.08%, respectively. This contrasts with the behavior of sandboxes, which often introduce more execution overhead and are therefore more susceptible to detection via timing-based checks. The high readiness of debuggers in these categories reflects their minimal impact on program execution timing, reducing their detectability through performance-based heuristics.

Table 5 further supports this observation, indicating that debuggers are generally well-prepared to counter miscellaneous evasive techniques implemented by packers. Although *NtQuerySystemTime* evaded detection in all tested debuggers, its use among packers is rare. Overall, the results suggest that most packers do not rely on Timing or Stalling techniques to evade debuggers—likely because debuggers introduce minimal runtime artifacts, making such checks less effective.

6.3.4 Combination of Packers' Evasive Techniques. Table 14 reports the results of testing combinations of techniques from the analyzed packers against the tested debuggers. Debuggers exhibit high evasion rates, with each debugger being evaded by 96% of the tested packers. This suggests that

debugger-based analysis offers limited resilience against packed samples, regardless of the specific tool employed. This result is consistent with the findings reported in Table 3. Packers heavily rely on anti-debugging techniques, enabling them to easily identify debugging environments before the execution of the actual malicious payload.

Answer to RQ3 and RQ4: Debuggers demonstrate to not be ready to counteract anti-debugging evasive techniques, with an average of 29% of countered techniques. Moreover, these techniques are highly effective and widely used by packers, highlighting the significant vulnerability of debuggers to packed malware. When considering combinations of evasive techniques as implemented by commercial packers, debugger resilience degrades even further. Our results show that each tested debugger is evaded by 96% of the packers, indicating that debugger-based analysis offers very limited protection against real-world packed samples, regardless of the specific tool employed. In isolation, debuggers appear generally effective at countering anti-virtualization techniques, with most categories showing high readiness. The average percentage of countered techniques is 80%. However, evasive methods based on locale checks, file system inspection, and virtual machine trap instructions remain both effective and used by packers to bypass analysis. Finally, debuggers are well-prepared to counter miscellaneous evasive techniques, particularly those involving timing and stalling. The percentage of countered technique is 84.6%. The few techniques that remain effective, such as specific registry and system environment checks, are rarely employed by packers, further reinforcing the robustness of debuggers in this category.

7 Related Works

Understanding evasive malware behaviors has long been a key objective in the security research community. Early efforts primarily focused on systematizing such behaviors or detecting them through static analysis. Afianian et al. [2] and Bulazel and Yener [9] provided comprehensive taxonomies of evasive techniques, highlighting how malware can detect analysis environments via virtual machine fingerprinting, timing discrepancies, or debugger presence. However, their studies are qualitative in nature and do not assess the real-world prevalence or effectiveness of these techniques.

Complementary to these efforts, Barbosa and Branco [6] and Chen et al. [12] employed large-scale static analysis to identify anti-debugging and anti-virtualization techniques in malware samples. Yet, these static approaches are inherently limited, as they cannot guarantee that identified code paths are ever executed, and are vulnerable to obfuscation and packing techniques.

Galloro et al. [24] addressed these limitations with a comprehensive longitudinal study of evasive techniques in Windows malware, analyzing 45,375 malware and 949 goodware samples from 2010 to 2019. Their dynamic analysis revealed a strong correlation between malware families and their preferred evasive strategies, providing the first large-scale behavioral perspective on evasive malware.

In a similar direction, D'Elia et al.[20] conducted a systematic analysis of anti-analysis behaviors and showed that manual analysts are routinely exposed to evasive techniques previously assumed to affect only automated systems. Their evaluation demonstrates how many evasions are still effective even when samples are manually dissected. Polino et al. [52] measured the prevalence of anti-instrumentation techniques across 7,006 malware samples, showing that over 15% actively attempt to detect runtime analysis. Their results highlight the widespread adoption of evasive logic even in commodity packers and obfuscated samples.

Miramirkhani et al. [45] introduced a novel class of sandbox evasion techniques based on the analysis of wear-and-tear artifacts, which capture indirect evidence of long-term user and system activity (e.g., estimating the age and usage history of a Windows installation). By extracting a

Table 15. Comparison of Large-Scale Studies on Evasive Malware Techniques

Work	Year	Taxonomy	# Evasive Techniques	Debuggers	Sandboxes	Packers	Dynamic Analysis	Goodware Comparison	Real-World Eval
Barbosa and Branco [6]	2014	✕	16	✕	✕	✕	✕	✕	✕
Chen et al. [12]	2016	✕	6	✕	✕	✕	✕	✕	✕
Polino et al. [52]	2017	✕	-	✕	✕	✕	✓	✓	✓
Bulazel and Yener [9]	2017	✓	41	✕	✕	✕	✕	✕	✕
Miramirkhani et al. [45]	2017	✓	44	✕	•	✕	✓	✓	✓
Afianian et al. [2]	2019	✓	41	✕	✕	✕	✕	✕	✕
D'Elia et al. [20]	2020	✕	-	✕	✕	✕	✓	✕	✓
Nappa et al.[48]	2021	✕	-	✕	•	✕	✓	✕	✓
Galloro et al. [24]	2022	✓	92	✕	✕	✕	✓	✓	✕
Yin et al. [77]	2023	✓	-	✕	✕	✕	✓	✕	✕
Xu et al. [75]	2023	✓	-	✕	✕	✕	✓	✕	✕
Ruggia et al. [60]	2024	✕	97	✕	✕	•	✓	✕	✕
This Work	2025	✓	**191**	•	•	•	✓	✓	✓

The **Taxonomy** column indicates whether the study proposed a classification of evasion methods. The **# Evasive Techniques** column reports the number of distinct evasive techniques analyzed. The **Debuggers** and **Sandboxes** columns indicate whether the study assessed the resilience of debuggers and sandboxes, respectively. The **Packers** column shows whether the study analyzed evasive behaviors introduced by packers. The **Dynamic Analysis**, **Goodware**, and **Real-World Eval** columns indicate whether the study performed dynamic testing, included goodware comparison, or tested real-world tools. Symbols: ✓ = provided, ✕ = not provided; • = addressed in detail, ◦ = partially addressed.

diverse set of such artifacts and training decision-tree classifiers, they showed that malware can distinguish real user systems from analysis environments with an accuracy of 92.86%.

Nappa et al. [48] introduced a novel sandbox evasion technique based on the execution of **Proof-of-Work** (**PoW**) algorithms. By exploiting the stable and hardware-dependent execution time of PoW computations, malware can detect virtualized analysis environments and suppress its malicious behavior. Their evaluation shows that embedding PoW significantly reduces the detection rate of real malware, while remaining difficult to flag as malicious, since identifying PoW computations would introduce a large number of false positives for benign applications that legitimately employ such algorithms.

On Android, Ruggia et al. [60] conducted a similar systematization and introduced *DroidDungeon*, a sandbox designed to bypass evasion tactics. Their work showed that 14% of Android malware terminates execution in the presence of analysis artifacts, and that evasive malware tends to interact more frequently with high-risk APIs, such as those invoking CLI commands.

Beyond systematization, recent studies have explored detection strategies for evasive malware. Yin et al. [77] proposed a classification method based on hierarchical behavior models derived from API traces, showing improved resilience to evasion through sequence-aware vectorization. Xu et al. [75] introduced *Antitoxin*, a taint-guided multi-path exploration framework capable of uncovering logic bombs and unknown evasive behaviors more effectively than traditional linear analyses.

In contrast to these efforts, our work provides a dual contribution: first, we evaluate how commercial and publicly available packers introduce evasive techniques into protected binaries; second, we test the readiness of real-world sandboxes and debuggers to counter these techniques. By correlating the evasive strategies injected by packers with the capabilities of analysis tools, we offer the first empirical study that bridges evasive obfuscation and analysis tool effectiveness. A summary of related works is provided in Table 15.

8 Conclusions

In this work, we conducted an experimental evaluation of the readiness of analysis environments, specifically sandboxes and debuggers, against evasive techniques. First, we propose an updated

taxonomy of evasive techniques, starting from the one proposed by Galloro et al. [24], with 55 newly discovered and previously undocumented techniques. Successively, to assess the environments readiness to such techniques, we studied the evasive techniques employed by 25 publicly available packers, and tested the readiness of 7 publicly available sandboxes and 4 debuggers. Our analysis of 25 packers reveals that evasive techniques are widely used during binary obfuscation. Anti-debugging methods were the most prevalent, followed by anti-virtualization and miscellaneous techniques. Among anti-debugging methods, *Exception Handling* and *Process Environment* queries were the most used. Anti-virtualization techniques focused on *CPU Fingerprinting*, *System Environment*, and *File System* inspection. In the miscellaneous category, timing-based techniques dominated, indicating that many packers attempt to detect analysis overhead introduced by emulation or instrumentation. However, some advanced evasion techniques—such as human interaction or locale-based checks—were implemented only by a small subset of packers. Our evaluation of seven publicly available sandboxes demonstrates high readiness against anti-debugging (72%) and anti-virtualization (85%) techniques. In particular, techniques commonly used by packers—such as IsDebuggerPresent and NtQueryInformationProcess—were effectively neutralized by most sandboxes. This suggests that sandboxes are well-prepared to handle the evasion strategies adopted by off-the-shelf packers. However, readiness dropped significantly in the miscellaneous category (42%), especially for timing and stalling-based techniques. In contrast to sandboxes, the four tested debuggers showed limited resilience to anti-debugging techniques, with only 29% of techniques countered. This aligns closely with the packer results: many packers employ direct anti-debugging methods, all of which were highly effective against debuggers. However, debuggers demonstrated strong resilience against anti-virtualization (80%) and miscellaneous (84.6%) techniques, including those most frequently used by packers. This is likely due to their native execution model, which minimizes behavioral and timing artifacts. Overall, while packers succeed in defeating debuggers through standard anti-debugging tactics, their effectiveness against debuggers in virtualization and timing-based evasion is limited. Our evaluation, while extensive, presents several limitations. Many commercial packers were only available in trial versions that might lack advanced features such as delayed execution or virtualization-aware checks. Another limitation arises from our decision to test each evasive technique in isolation. While this facilitates clear attribution of observed effects, it does not reflect real-world scenarios where malware typically combines multiple evasion strategies. Lastly, our analysis is based on a minimal "Hello World" binary, which, despite its reproducibility, cannot fully capture the complexity and richness of actual malware behaviors.

Our empirical findings suggest that while analysis environments are effective against common artifacts, they lack a comprehensive approach to concealing architectural characteristics. Based on the observed vulnerabilities, we propose the following design changes:

- **Semantic consistency over artifact removal**: Sandboxes often fail to simulate realistic user activity, which makes them detectable due to the absence of typical system artifacts. Defensive designs should, therefore, move beyond minimal or freshly installed operating systems and incorporate realistic user interactions, such as mouse movement and keyboard input, in order to better resemble real user environments.
- **Transparent Signal Handling:** The low readiness of debuggers against exception-based evasion (9.09%) and traps (11.11%) indicates that these tools often interfere with the program's intended control flow. Defensive designs should therefore ensure that exceptions and privileged instructions are forwarded transparently to the analyzed program, preventing detection through signal interception.
- **Timing Jitter and Side-Channel Normalization:** Timing-based techniques are the most prevalent miscellaneous evasion (63.3%) used by packers to detect the overhead introduced by virtualization. Analysis environments should implement timing normalization for

hypervisor-sensitive instructions, such as CPUID, in order to reduce the latency differences caused by VM-exit transitions.

— **Out-of-Process Introspection:** Many evasive checks target user-space structures such as the **Process Environment Block** (**PEB**) or KUSER_SHARED_DATA to detect debugging artifacts. To mitigate this, defensive tools can rely on kernel-mode introspection or dynamic recompilation techniques, which execute rewritten code in separate memory regions and reduce exposure to memory fingerprinting and integrity checks.

This work opens several avenues for future research. A key direction is to analyze the effect of combining multiple evasive techniques within the same binary, to better reflect the complexity of modern malware. Expanding the scope of our analysis to include 64-bit-exclusive evasions and non-Windows platforms such as Linux and Android would also provide a more comprehensive assessment of analysis environment readiness. Furthermore, a longitudinal study on the evolution of commercial packers could shed light on emerging trends in evasion. Another promising area is the design of improved anti-evasion mechanisms that focus on obfuscating timing artifacts, simulating user activity, and concealing analysis-related system features. Finally, the development of a standardized benchmarking framework for evasive techniques would facilitate ongoing evaluation and comparison of analysis tools in both research and industry contexts.

References

[1] Amir Afianian, Salman Niksefat, Babak Sadeghiyan, and David Baptiste. 2019. Malware dynamic analysis evasion techniques: A survey. *ACM Computing Surveys (CSUR)* 52, 6 (2019), 1–28.

[2] Amir Afianian, Salman Niksefat, Babak Sadeghiyan, and David Baptiste. 2019. Malware dynamic analysis evasion techniques: A survey. *ACM Comput. Surv.* 52, 6, Article 126 (Nov. 2019), 28 pages.

[3] Frances E Allen. 1970. Control flow analysis. *ACM Sigplan Notices* 5, 7 (1970), 1–19.

[4] ANY.RUN [n. d.]. *ANY.RUN — Interactive Malware Sandbox.* ANY.RUN. Retrieved January 27, 2026 from https://app.any.run/

[5] John Aycock. 2006. *Computer Viruses and Malware.* Advances in Information Security, Vol. 22. Springer. DOI: https://doi.org/10.1007/0-387-34188-9

[6] Gabriel Negreira Barbosa and Rodrigo Rubira Branco. 2014. Prevalent characteristics in modern malware. In *Black Hat USA*. Las Vegas, NV.

[7] M. V. Belyaev, N. V. Shimchik, V. N. Ignatyev, and A. A. Belevantsev. 2018. Comparative analysis of two approaches to static taint analysis. *Programming and Computer Software* 44 6 (2018), 459–466.

[8] Danilo Bruschi, Lorenzo Martignoni, and Mattia Monga. 2006. Detecting self-mutating malware using control-flow graph matching. In *Detection of Intrusions and Malware & Vulnerability Assessment, Third International Conference, DIMVA 2006, Berlin, Germany, July 13-14, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4064)*, Roland Büschkes and Pavel Laskov (Eds.). Springer, 129–143. DOI: https://doi.org/10.1007/11790754_8

[9] Alexei Bulazel and Bülent Yener. 2017. A survey on automated dynamic malware analysis evasion and counter-evasion: PC, mobile, and web. In *Proceedings of the 1st Reversing and Offensive-Oriented Trends Symposium* (Vienna, Austria) *(ROOTS)*. Association for Computing Machinery, New York, NY, USA, Article 2, 21 pages.

[10] C2AE [n. d.]. *C2AE Malware Analysis Sandbox.* C2AE. Retrieved January 27, 2026 from https://docs.virustotal.com/docs/external-sandboxes

[11] Luca Caviglione, Michał Choraś, Igino Corona, Artur Janicki, Wojciech Mazurczyk, Marek Pawlicki, and Katarzyna Wasielewska. 2020. Tight arms race: Overview of current malware threats and trends in their detection. *IEEE Access* 9 (2020), 5371–5396.

[12] Ping Chen, Christophe Huygens, Lieven Desmet, and Wouter Joosen. 2016. Advanced or not? a comparative study of the use of anti-debugging and anti-vm techniques in generic and targeted malware. In *ICT Systems Security and Privacy Protection*, Jaap-Henk Hoepman and Stefan Katzenbeisser (Eds.). Springer International Publishing, Cham, 323–336.

[13] Mario Ciccarelli. 2019. Going Deep: A Guide to Reversing Smoke Loader Malware. Retrieved March 6, 2025 from https://www.sentinelone.com/blog/going-deep-a-guide-to-reversing-smoke-loader-malware/

[14] Microsoft Corporation. 2023. *WinDbg: Windows Debugger.* Retrieved March 5, 2025 from https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/debugger-download-tools

[15] Cyberbit. 2007. Anti-VM and Anti-Sandbox Explained. Retrieved April 16, 2025 from https://www.cyberbit.com/blog/endpoint-security/anti-vm-and-anti-sandbox-explained/

[16] Cybereason Nocturnus. 2021. Cybereason vs. DarkSide Ransomware. Retrieved from https://www.cybereason.com/blog/research/cybereason-vs-darkside-ransomware
[17] CyberMasterV. 2022. A Detailed Analysis of Lazarus APT Malware Disguised as Notepad++ Shell Extension. Retrieved March 6, 2025 from https://cybergeeks.tech/a-detailed-analysis-of-lazarus-malware-disguised-as-notepad-shell-extension/
[18] Deep Instinct. 2019. Anti-Virtualization Malware. Retrieved April 16, 2025 from https://www.deepinstinct.com/2019/10/29/malware-evasion-techniques-part-2-anti-vm-blog/
[19] Gabriele Digregorio, Roberto Alessandro Bertolini, Francesco Panebianco, and Mario Polino. 2024. *libdebug: Build Your Own Debugger*. DOI: https://doi.org/10.5281/zenodo.13151549
[20] Daniele Cono D'Elia, Emilio Coppa, Federico Palmaro, and Lorenzo Cavallaro. 2020. On the dissection of evasive malware. *IEEE Transactions on Information Forensics and Security* 15 (2020), 2750–2765.
[21] Jannatul Ferdous, Rafiqul Islam, Arash Mahboubi, and Md Zahidul Islam. 2023. A review of state-of-the-art malware attack trends and defense mechanisms. *IEEe Access* 11 (2023), 121118–121141.
[22] Peter Ferrie. 2011. The Ultimate Anti-Debug Reference. Retrieved April 16, 2025 from https://anti-reversing.com/Downloads/Anti-Reversing/The_Ultimate_Anti-Reversing_Reference.pdf
[23] Sai Omkar Vashisht and Abhishek Singh. 2014. Turing test in reverse: New sandbox-evasion techniques seek human interaction. FireEye Threat Research Blog. Retrieved June 24, 2014 from https://web.archive.org/web/20220517173924/https://www.fireeye.com/blog/threat-research/2014/06/turing-test-in-reverse-new-sandbox-evasion-techniques-seek-human-interaction.html
[24] Nicola Galloro, Mario Polino, Michele Carminati, Andrea Continella, and Stefano Zanero. 2022. A systematical and longitudinal study of evasive behaviors in windows malware. *Comput. Secur.* 113 (2022), 102550. DOI: https://doi.org/10.1016/J.COSE.2021.102550
[25] GNU Project. 2025. *GDB: The GNU Project Debugger*. Free Software Foundation. Retrieved April 22, 2025 from https://www.gnu.org/software/gdb/
[26] Immunity Inc. 2023. *Immunity Debugger*. Retrieved March 3, 2025 from https://www.immunityinc.com/products/debugger
[27] Dalya Guttman. 2017. Common anti-debugging techniques in the malware landscape. Deep Instinct. Retrieved July 9, 2026 from https://web.archive.org/web/20260709144059/https://www.deepinstinct.com/blog/common-anti-debugging-techniques-in-the-malware-landscape
[28] Intezer [n. d.]. *Intezer Analyze — Malware Analysis Platform*. Intezer. Retrieved January 27, 2026 from https://analyze.intezer.com/
[29] Joesandbox Report 2016. Retrieved from https://www.joesecurity.org/blog/6933341622592617830
[30] JoeSandbox Report. 2019. JoeSandbox Report. Retrieved March 6, 2025 from https://www.joesecurity.org/reports/report-c5e1106f9654a23320132cbc61b3f29d.html
[31] JoeSandbox Report. 2020. Analyzing Azorult's Anti-Analysis Tricks with Joe Sandbox Hypervisor. Retrieved March 6, 2025 from https://www.joesecurity.org/blog/9048980422564630717
[32] JoeSandbox Report. 2020. New Sandbox Evasions Spotted in VBS Samples. Retrieved March 6, 2025 from https://www.joesecurity.org/blog/745788703303246608
[33] Mohammad Sina Karvandi, MohammadHosein Gholamrezaei, Saleh Khalaj Monfared, Soroush Meghdadi Zanjani, Behrooz Abbassi, Ali Amini, Reza Mortazavi, Saeid Gorgin, Dara Rahmati, and Michael Schwarz. 2022. HyperDbg: Reinventing hardware-assisted debugging. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 1709–1723. DOI: https://doi.org/10.1145/3548606.3560649
[34] Clemens Kolbitsch, Engin Kirda, and Christopher Kruegel. 2011. The power of procrastination: detection and mitigation of execution-stalling malicious code. In *Proceedings of the 18th ACM Conference on Computer and Communications Security, CCS 2011, Chicago, Illinois, USA, October 17-21, 2011*, Yan Chen, George Danezis, and Vitaly Shmatikov (Eds.). ACM, 285–296. DOI: https://doi.org/10.1145/2046707.2046740
[35] Oleg Kulchytskyy and Anton Kukoba. 2019. Anti debugging protection techniques with examples. Apriorit. Retrieved July 9, 2026 from https://web.archive.org/web/20260709225316/https://www.apriorit.com/dev-blog/367-anti-reverse-engineering-protection-techniques-to-use-before-releasing-software
[36] Martina Lindorfer, Clemens Kolbitsch, and Paolo Milani Comparetti. 2011. Detecting environment-sensitive malware. In *Recent Advances in Intrusion Detection: 14th International Symposium, RAID 2011, Menlo Park, CA, USA, September 20-21, 2011. Proceedings 14*. Springer, 338–357.
[37] LLVM Project. 2025. *The LLDB Debugger*. LLVM Project. Retrieved April 22, 2025 from https://lldb.llvm.org/
[38] Juan Lopez, Leonardo Babun, Hidayet Aksu, and A. Selcuk Uluagac. 2017. A survey on function and system call hooking approaches. *Journal of Hardware and Systems Security* 1, 2 (2017), 114–136.
[39] LordNoteworthy. n.d.. Al-khaser. Retrieved March 6, 2025 from https://github.com/LordNoteworthy/al-khaser

[40] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: Building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery (ACM), 190–200. DOI: https://doi.org/10.1145/1065010.1065034

[41] Malware.News. 2017. Malware Evasion Techniques: Same Wolf—Different Clothing. Retrieved March 6, 2025 from https://malware.news/t/malware-evasion-techniques-same-wolf-different-clothing/14718

[42] Malware.News. Evolution of Malware Sandbox Evasion Tactics—A Retrospective Study 2019. Retrieved from https://malware.news/t/evolution-of-malware-sandbox-evasion-tactics-a-retrospective-study/32820

[43] McAfee. 2019. Evolution of Malware Sandbox Evasion Tactics: A Retrospective Study. Retrieved April 16, 2025 from https://securingtomorrow.mcafee.com/other-blogs/mcafee-labs/evolution-of-malware-sandbox-evasion-tactics-a-retrospective-study/

[44] Xavier Mertens. 2019. Keep an Eye on Your WMI Logs. Retrieved March 6, 2025 from https://isc.sans.edu/diary/Keep+an+Eye+on+Your+WMI+Logs/25012

[45] Najmeh Miramirkhani, Mahathi Priya Appini, Nick Nikiforakis, and Michalis Polychronakis. 2017. Spotless sandboxes: Evading malware analysis systems using wear-and-tear artifacts. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1009–1024.

[46] Andreas Moser, Christopher Kruegel, and Engin Kirda. 2007. Limits of static analysis for malware detection. In *23rd Annual Computer Security Applications Conference (ACSAC 2007), December 10-14, 2007, Miami Beach, Florida, USA*. IEEE Computer Society, 421–430. DOI: https://doi.org/10.1109/ACSAC.2007.21

[47] Alexandre Mundo. 2019. Clop Ransomware. Retrieved March 6, 2025 from https://www.mcafee.com/blogs/other-blogs/mcafee-labs/clop-ransomware

[48] Antonio Nappa, Panagiotis Papadopoulos, Matteo Varvello, Daniel Aceituno Gomez, Juan Tapiador, and Andrea Lanzi. 2021. PoW-How: An enduring timing side-channel to evade online malware sandboxes. In *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4-8, 2021, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 12972)*, Elisa Bertino, Haya Schulmann, and Michael Waidner (Eds.). Springer, 86–109. DOI: https://doi.org/10.1007/978-3-030-88418-5_5

[49] Nicholas Nethercote and Julian Seward. 2007. Valgrind: A framework for heavyweight dynamic binary instrumentation. In *Proceedings of the ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery (ACM), 89–100. DOI: https://doi.org/10.1145/1273442.1250746

[50] Yoshihiro Oyama. 2018. Investigation of the diverse sleep behavior of malware. *Journal of Information Processing* 26 (2018), 461–476.

[51] Pikker (Cuckoo Sandbox) [n. d.]. *Pikker / Cuckoo Sandbox Instance*. Pikker (Cuckoo Sandbox). Retrieved January 27, 2026 from https://sandbox.pikker.ee/

[52] Mario Polino, Andrea Continella, Sebastiano Mariani, Stefano D'Alessio, Lorenzo Fontana, Fabio Gritti, and Stefano Zanero. 2017. Measuring and defeating anti-instrumentation-equipped malware. In *Detection of Intrusions and Malware, and Vulnerability Assessment—14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings (Lecture Notes in Computer Science, Vol. 10327)*, Michalis Polychronakis and Michael Meier (Eds.). Springer, 73–96. DOI: https://doi.org/10.1007/978-3-319-60876-1_4

[53] Qianxin [n. d.]. *Qianxin Sandbox*. Qianxin. Retrieved January 27, 2026 from https://sandbox.qianxin.com/

[54] Ole André V. Ravnås. 2025. Frida: Dynamic Instrumentation Toolkit. Retrieved April 22, 2025 from https://frida.re/

[55] Yaraslau Harakhavik. 2020. Anti-Debug Tricks. Check Point Research. Retrieved September 16, 2026 from https://anti-debug.checkpoint.com/

[56] Raman Ladutska. 2020. Evasion Techniques. Check Point Research. Retrieved September 16, 2026 from https://evasions.checkpoint.com/

[57] Thomas Roccia. 2023. Checking Screen Resolution. Retrieved March 6, 2025 from https://unprotect.it/technique/checking-screen-resolution/

[58] Thomas Roccia. 2023. Connected Printer. Retrieved March 6, 2025 from https://unprotect.it/technique/connected-printer/

[59] Thomas Roccia. 2023. Detecting USB Drive. Retrieved March 6, 2025 from https://unprotect.it/technique/detecting-usb-drive/

[60] Antonio Ruggia, Dario Nisi, Savino Dambra, Alessio Merlo, Davide Balzarotti, and Simone Aonzo. 2024. Unmasking the veiled: A comprehensive analysis of android evasive malware. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) *(ASIA CCS '24)*. Association for Computing Machinery, New York, NY, USA, 383–398. DOI: https://doi.org/10.1145/3634737.3637658

[61] Donato Onofri Sarang Sonawane. 2022. *Malware Analysis: GuLoader Dissection Reveals New Anti-Analysis Techniques and Code Injection Redundancy*. Technical Report. Crowdstrike.

[62] SecureWorks. 2019. REvil/Sodinokibi Ransomware. Retrieved March 6, 2025 from https://www.secureworks.com/research/revil-sodinokibi-ransomware

[63] SecureWorks. 2022. DarkTortilla Malware Analysis. Retrieved March 6, 2025 from https://www.secureworks.com/research/darktortilla-malware-analysis
[64] Michael Sikorski and Andrew Honig. 2012. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. no starch press.
[65] Abhishek Singh and Zheng Bu. 2021. Hot Knives Through Butter: Evading File-based Sandboxes. Retrieved March 6, 2025 from https://web.archive.org/web/20220313062547/https://www.mandiant.com/sites/default/files/2021-09/fireeye-hot-knives-through-butter.pdf
[66] Jagsir Singh and Jaswinder Singh. 2021. A survey on machine learning-based malware detection in executable files. *Journal of Systems Architecture* 112 (2021), 101861.
[67] Venkatachalabathy SR. 2016. Locky Ransomware on Rampage With JavaScript Downloader. Retrieved March 6, 2025 from https://www.mcafee.com/blogs/other-blogs/mcafee-labs/locky-ransomware-rampage-javascript-downloader
[68] Brett Stone-Gross and Nikolaos Pantazopoulos. 2023. Technical Analysis of Pikabot. Retrieved from https://www.zscaler.com/blogs/security-research/technical-analysis-pikabot
[69] Bill Toulas. 2023. New Stealthy 'Beep' Malware Focuses Heavily on Evading Detection. Retrieved March 6, 2025 from https://www.bleepingcomputer.com/news/security/new-stealthy-beep-malware-focuses-heavily-on-evading-detection
[70] Hatching / Recorded Future [n. d.]. *Triage — Malware Analysis Sandbox*. Hatching / Recorded Future. Retrieved January 27, 2026 from https://tria.ge/
[71] Xabier Ugarte-Pedrero, Davide Balzarotti, Igor Santos, and Pablo García Bringas. 2015. SoK: Deep packer inspection: A longitudinal study of the complexity of run-time packers. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*. IEEE Computer Society, 659–673. DOI: https://doi.org/10.1109/SP.2015.46
[72] Unprotect Project. 2017. Anti-Debugging. Retrieved April 16, 2025 from http://unprotect.tdgt.org/index.php/Anti-debugging
[73] Mihai Vasilescu, Laura Gheorghe, and Nicolae Tapus. 2014. Practical malware analysis based on sandboxing. In *2014 RoEduNet Conference 13th Edition: Networking in Education and Research Joint Event RENAM 8th Conference*. 1–6. DOI: https://doi.org/10.1109/RoEduNet-RENAM.2014.6955304
[74] x64dbg Developers. 2023. *x64dbg: Open-source Debugger for Windows*. Retrieved March 3, 2025 from https://x64dbg.com
[75] Fangzhou Xu, Wang Zhang, Weizhong Qiang, and Hai Jin. 2023. Multi-path exploration guided by taint and probability against evasive malware. *Secur. Saf.* 2 (2023), 2023023. DOI: https://doi.org/10.1051/SANDS/2023023
[76] Jiaqi Yan, Guanhua Yan, and Dong Jin. 2019. Classifying malware represented as control flow graphs using deep graph convolutional neural network. In *49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2019, Portland, OR, USA, June 24-27, 2019*. IEEE, 52–63. DOI: https://doi.org/10.1109/DSN.2019.00020
[77] Haikuo Yin, Brandon Lou, and Peter Reiher. 2023. A method for summarizing and classifying evasive malware. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses* (Hong Kong, China) *(RAID '23)*. Association for Computing Machinery, New York, NY, USA, 455–470. DOI: https://doi.org/10.1145/3607199.3607207
[78] Oleh Yuschuk. 2023. *OllyDbg: 32-bit Assembler-level Debugger for Windows*. Retrieved March 3, 2025 from http://www.ollydbg.de
[79] Lenny Zeltser. 2024. Free Automated Malware Analysis Sandboxes and Services. Retrieved May 9, 2025 from https://zeltser.com/automated-malware-analysis/
[80] Qinghua Zhang and Douglas S. Reeves. 2007. MetaAware: Identifying metamorphic malware. In *23rd Annual Computer Security Applications Conference (ACSAC 2007), December 10-14, 2007, Miami Beach, Florida, USA*. IEEE Computer Society, 411–420. DOI: https://doi.org/10.1109/ACSAC.2007.9

Received 6 June 2025; revised 7 March 2026; accepted 2 September 2026